

OpenClaw in azienda

Sistema multi-utente con isolamento per ruolo

L'accesso ai dati è imposto dal database e dal sistema operativo, non dalle istruzioni date all'intelligenza artificiale.

Destinatario: chi installa, configura e consegna il sistema presso il cliente

Ambiente di riferimento: server Linux Ubuntu LTS dedicato, sempre acceso

REGOLA Questa guida

Documento operativo per l'installazione di un assistente AI aziendale multi-utente su un unico server. Ogni persona accede soltanto ai dati della propria mansione, e il divieto è imposto dal database e dal sistema operativo, non dalle istruzioni date all'intelligenza artificiale. Tutte le impostazioni riportate sono state verificate sulla documentazione ufficiale OpenClaw, elencata nell'Allegato L.

Come è organizzata. La Parte I spiega come funziona il sistema e perché è costruito così: leggila prima, anche se hai fretta, perché tutte le scelte delle fasi successive dipendono da lì. La Parte II contiene le decisioni da prendere insieme al cliente. La Parte III è l'installazione vera e propria, fase per fase. La Parte IV è il collaudo e la gestione ordinaria.

Come si legge una fase. Ogni fase ha sempre quattro sezioni: **a cosa serve** in linguaggio piano, **i comandi esatti** nell'ordine in cui vanno eseguiti, **che cosa significano le impostazioni** che hai appena scritto, e una **Verifica** con il risultato che devi ottenere. Se una verifica non dà il risultato atteso, fermati e non passare oltre: le verifiche sono la differenza fra un sistema sicuro e un sistema che sembra sicuro.

Indice

Parte I — Come funziona il sistema

1. Che cosa stai installando
2. I pezzi del sistema e come si chiamano
3. Il percorso di un messaggio, dall'inizio alla fine
4. Lo schema dell'architettura
5. Dove sta davvero la sicurezza
6. Le tre regole del modello di fiducia

Parte II — Decisioni da prendere con il cliente

7. Quanti gateway servono
8. Fase 0 — La matrice degli accessi
9. Come entrano i dipendenti
10. Quale modello AI
11. Cosa procurarsi dal cliente prima di iniziare
12. La scheda d'impianto e l'albero delle decisioni

Parte III — Installazione passo passo

- Fase 1 — Preparazione del server e firewall
- Fase 2 — Un utente Linux per ruolo
- Fase 3 — Cartelle dei dati, permessi e ACL
- Fase 4 — PostgreSQL: ruoli, viste e riservatezza per colonna
- Fase 5 — L'immagine della sandbox
- Fase 6 — Installazione del gateway
- Fase 7 — Configurazione del gateway
- Fase 8 — Agente, tool policy e isolamento delle sessioni
- Fase 9 — Sandbox e cartelle montate
- Fase 10 — Accesso al database tramite MCP
- Fase 11 — Canali di messaggistica e instradamento delle persone
- Fase 12 — Accesso da interfaccia web (opzionale)
- Fase 13 — Knowledge base condivisa
- Fase 14 — Segreti e permessi dei file
- Fase 15 — Registro append-only

Parte IV — Collaudo, consegna ed esercizio

- Fase 16 — Collaudo: i quattordici test negativi
- Fase 17 — Esercizio: onboarding, offboarding, aggiornamenti
- Fase 18 — Aggiornare senza rompere nulla
- Fase 19 — Procedura di incidente
- 20. Cosa non si può garantire

Allegati

- A — Scheda d'impianto da compilare
- B — Procedura parametrica: una mansione da zero
- C — Configurazione completa: mansione di riferimento
- D — Differenze per le altre mansioni
- E — Variante senza database: solo cartelle
- F — Reverse proxy: Caddy e nginx
- G — Installazione a celle con openclaw fleet
- H — Dizionario delle impostazioni
- I — Diagnostica: sintomo, causa, comando
- J — Checklist di consegna

- K — Tabella di risposta rapida al cliente
- L — Fonti ufficiali utilizzate

PARTE I — COME FUNZIONA IL SISTEMA

1. Che cosa stai installando

Il cliente vuole un assistente che risponda su una chat aziendale e che sappia rispondere sui dati veri dell'azienda: fatture, clienti, contratti, procedure interne. Il problema non è farlo funzionare: è farlo funzionare **senza che la segretaria possa vedere gli stipendi** e senza che il titolare debba fidarsi della buona volontà di un programma.

Il pezzo che va capito prima di tutto

OpenClaw è un programma che riceve un messaggio, lo passa a un modello di intelligenza artificiale e mette a disposizione del modello degli strumenti: leggere un file, scrivere un file, eseguire un comando, interrogare un database. Quando il modello decide di usare uno strumento, OpenClaw lo esegue davvero, sul server, con l'identità del processo che lo ospita.

Da qui discende tutto il resto. Se scrivi in un file di istruzioni "non leggere la cartella amministrazione", quella è una **richiesta di comportamento**: il modello può ignorarla, può essere manipolato da un testo che legge, o può semplicemente sbagliare. Se invece l'utente di sistema sotto cui gira quel programma non ha il permesso di leggere quella cartella, il sistema operativo risponde **Permission denied** e non c'è frase che lo cambi.

REGOLA Il principio dell'intera installazione
Non si chiede all'intelligenza artificiale di rispettare i confini. Si costruiscono i confini in modo che l'intelligenza artificiale non possa attraversarli nemmeno volendo.

Che cosa questo sistema è, e che cosa non è

È	Non è
Un assistente per persone che lavorano nella stessa azienda, ognuna con la propria mansione	Un servizio multi-utente per persone che non si fidano l'una dell'altra
Un sistema in cui i permessi sono quelli del sistema operativo e del database	Un sistema in cui i permessi sono scritti in un prompt
Un'installazione su un server che il cliente controlla	Un servizio in cloud condiviso fra clienti diversi
Uno strumento che risponde con i dati che il database gli restituisce	Un motore di ricerca che "ricorda" i dati aziendali

2. I pezzi del sistema e come si chiamano

Sono nove nomi. Vanno conosciuti perché ricorrono in ogni file di configurazione e in ogni comando delle fasi successive.

Nome	Che cos'è, in concreto
Gateway	Il processo che sta sempre acceso e che tiene insieme tutto: riceve i messaggi dai canali, parla con il modello, esegue gli strumenti. Un gateway è il confine dell'installazione: tutto ciò che sta dentro un gateway condivide gli stessi poteri.
Agente	Una configurazione dentro il gateway: quale modello usare, quale cartella di lavoro, quali strumenti sono permessi, con quale personalità risponde. Un gateway può contenerne più di uno.
Canale	La via da cui arrivano i messaggi: un bot Telegram, un numero WhatsApp, l'interfaccia web. Ogni canale ha le proprie regole su chi è ammesso a scrivere.
Binding	La regola che dice quale agente deve rispondere a quale conversazione. Serve solo quando in un gateway ci sono più agenti.
Sessione	Il filo della conversazione, con la sua cronologia. Chi condivide una sessione condivide la cronologia: è il motivo per cui l'impostazione della Fase 8 non è facoltativa.
Strumento (tool)	Una capacità concreta che il modello può chiedere di usare: <code>read</code> legge un file, <code>write</code> lo scrive, <code>exec</code> esegue un comando di shell, un server MCP interroga il database. La tool policy decide quali esistono per quell'agente.
Sandbox	Un container in cui vengono eseguiti gli strumenti. Dentro la sandbox si vede solo ciò che le è stato montato esplicitamente: tutto il resto del server non esiste.
Workspace	La cartella di lavoro dell'agente, dove stanno i suoi file di istruzioni e i file che produce.
MCP	Il modo standard per collegare all'agente uno strumento esterno, per esempio un connettore verso PostgreSQL. Gli strumenti MCP vengono eseguiti dal gateway sull'host, non dentro la sandbox: è una proprietà che in questa architettura si sfrutta apposta (Fase 10).

NOTA Un solo gateway, un solo confine di fiducia

La configurazione multi-agente separa il **lavoro**: workspace diversi, modelli diversi, strumenti diversi. Non separa le **persone**: dentro un gateway l'accesso operatore autenticato è un ruolo di controllo fidato, non un ruolo per singolo utente. Per separare le persone servono gateway distinti. È il motivo per cui questa installazione ne prevede uno per ruolo.

3. Il percorso di un messaggio, dall'inizio alla fine

Questo è ciò che accade, nell'ordine, quando una dipendente scrive all'assistente. Ogni passaggio corrisponde a una fase dell'installazione: se sai leggere questo schema, sai anche dove intervenire quando qualcosa non funziona.

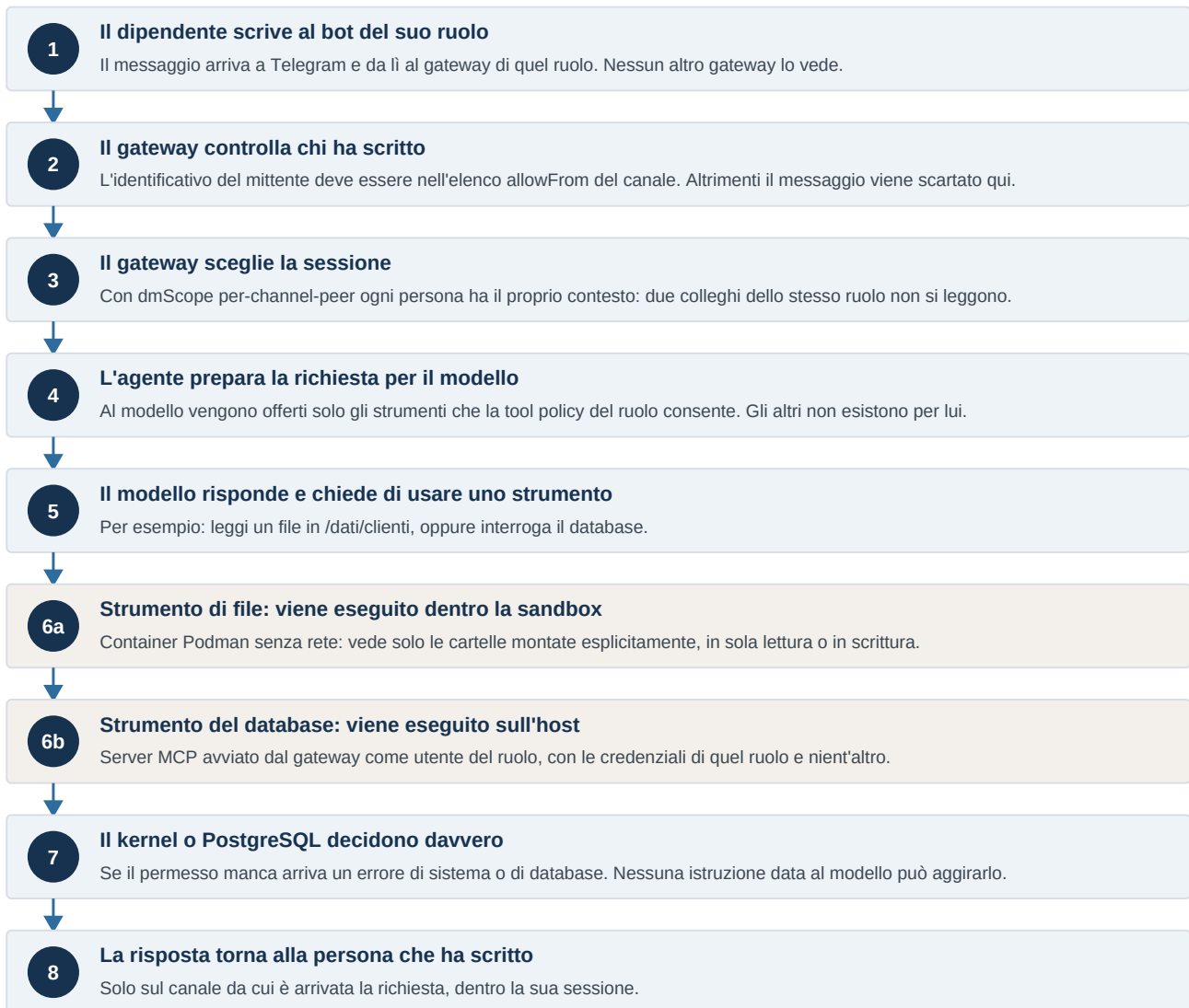


Figura 1 — Il percorso di una richiesta. I passaggi 6a e 6b sono alternativi e dipendono dallo strumento che il modello ha chiesto di usare.

Le tre porte che il messaggio deve attraversare

- **Prima porta, chi sei** (passaggio 2). L'elenco allowFrom del canale. Chi non è nell'elenco non riceve risposta: il messaggio non arriva nemmeno al modello. Si configura nella Fase 11.
- **Seconda porta, cosa puoi chiedere** (passaggio 4). La tool policy dell'agente. Se exec è negato, il modello non vede affatto quello strumento: non può quindi né usarlo né essere convinto a usarlo. Si configura nella Fase 8.
- **Terza porta, cosa puoi effettivamente toccare** (passaggi 6 e 7). I permessi del filesystem, i mount della sandbox e i privilegi del ruolo di database. È l'unica porta che regge anche se le prime due vengono aggirate. Si configura nelle Fasi 3, 4 e 9.

4. Lo schema dell'architettura

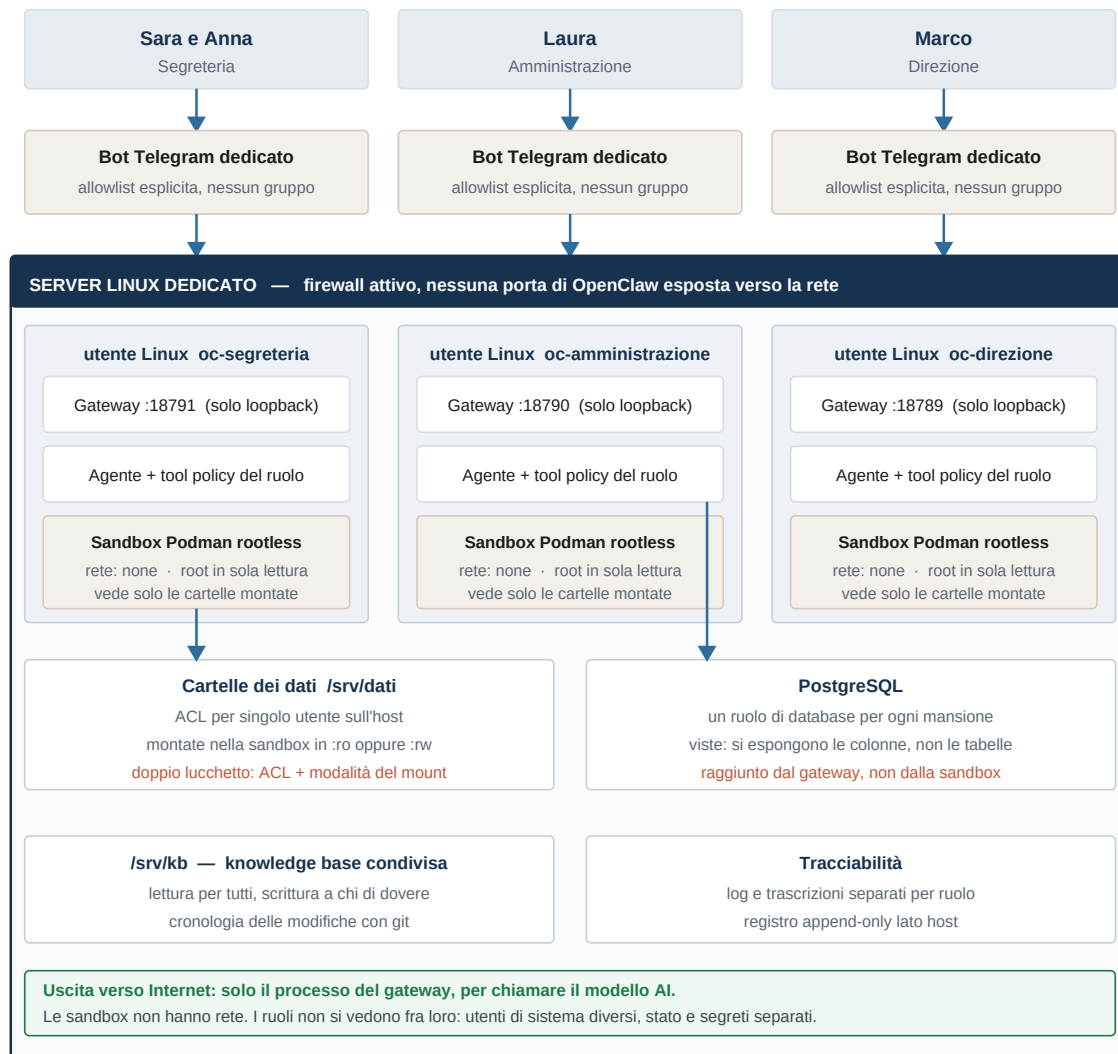


Figura 2 — Architettura completa su un unico server. Le frecce di esempio sono tracciate su un solo ruolo: la struttura è identica per gli altri.

Come si legge lo schema

- **Sopra il server** ci sono le persone e i canali. Ogni ruolo ha il proprio bot: il bot è la porta d'ingresso di quel confine di fiducia, e non viene mai condiviso fra ruoli.
- **Dentro il server**, ogni ruolo ha un utente Linux dedicato. Sotto quell'utente girano il suo gateway, il suo agente e la sua sandbox. Stato, segreti, cronologia delle conversazioni: tutto vive nella home di quell'utente, e nessun altro ruolo può leggerla.
- **Le porte dei gateway sono su loopback**, cioè raggiungibili solo dalla macchina stessa. Il firewall non espone nulla verso la rete.
- **Le cartelle dei dati sono fuori** dalle home dei ruoli, in `/srv/dati`. Ogni ruolo le vede solo se gliele monti nella sandbox, e solo nella modalità che hai indicato. Sull'host, in parallelo, le ACL concedono l'accesso solo agli utenti previsti: i due controlli si coprono a vicenda.
- **Il database è raggiunto dal gateway**, non dalla sandbox. Il connettore usa le credenziali del ruolo, quindi il database applica i propri permessi indipendentemente da cosa chiedi al modello.
- **L'unica uscita verso Internet** è quella del processo del gateway, che chiama il modello AI. Le sandbox non hanno rete: un agente non può inviare dati fuori nemmeno se manipolato.

5. Dove sta davvero la sicurezza

Cinque livelli, in ordine di robustezza. Più il livello è alto nell'elenco, più il divieto è difficile da aggirare.

LIVELLO 1 Il database	GRANT, viste, row level security Riservatezza fino alla singola colonna e riga
LIVELLO 2 Il kernel Linux	utenti, permessi, ACL Riservatezza su cartelle e file: Permission denied
LIVELLO 3 La sandbox	bind mount del container L'agente vede solo ciò che monta, in ro o rw
LIVELLO 4 OpenClaw	tool policy, allowlist, scope Riduce la superficie: quali strumenti e chi può parlare
LIVELLO 5 I file .md del workspace	SOUL.md, AGENTS.md Nessuna garanzia: sono indicazioni di comportamento

Figura 3 — I livelli di controllo. I livelli 1, 2 e 3 sono imposti da componenti che non eseguono istruzioni del modello. Il livello 4 è configurazione applicativa. Il livello 5 non è una garanzia.

REGOLA Regola operativa

Se un controllo che hai promesso al cliente esiste solo al livello 4 o 5, per il cliente non esiste. Ogni divieto scritto nel contratto deve avere una corrispondenza al livello 1, 2 o 3.

Perché i livelli si sovrappongono invece di sostituirsi

In questa installazione lo stesso divieto viene applicato più volte, da componenti diversi. La cartella delle fatture, per la segreteria, è negata dall'ACL sul filesystem, non è montata nella sandbox, e le colonne economiche non compaiono nella vista del database che quel ruolo può leggere. Tre controlli indipendenti per lo stesso divieto.

Non è ridondanza inutile: è la ragione per cui un errore di configurazione non diventa automaticamente una fuga di dati. Se domani, aggiornando la configurazione, monti per sbaglio quella cartella in scrittura, l'ACL del filesystem continua a bloccare. Questo va spiegato al cliente, perché è il motivo per cui il lavoro costa quello che costa.

6. Le tre regole del modello di fiducia

Sono i tre punti fermi dichiarati dalla documentazione ufficiale. Vanno riportati al cliente in fase di offerta, perché definiscono il perimetro di ciò che puoi promettere.

Regola 1 — Un gateway è un solo confine di fiducia

Il modello di sicurezza previsto è quello dell'assistente personale: un operatore fidato per gateway. OpenClaw non è un confine di sicurezza contro utenti reciprocamente ostili che condividono lo stesso agente o lo stesso gateway. Per operatività a fiducia mista i confini vanno separati con gateway e credenziali distinti, preferibilmente con utenti di sistema o host distinti. Questa installazione applica esattamente quella indicazione.

Fonte: docs.openclaw.ai/gateway/security

Regola 2 — L'identificativo di sessione non è un'autorizzazione

Gli identificativi di sessione, le etichette e le chiavi di conversazione sono selettori di instradamento: dicono **dove** va il messaggio, non **se** chi lo manda ne ha diritto. Di conseguenza l'identità su Telegram o WhatsApp è un instradamento, non un login: chi ha in mano il telefono sbloccato di un dipendente parla con l'agente di quel

dipendente. La protezione che regge in quel caso è quella del database e dei permessi.

Regola 3 — Il token del gateway è accesso pieno

L'autenticazione con segreto condiviso, cioè token o password del gateway, ripristina l'intero set di scope operatore predefiniti anche se il chiamante ne dichiara di più stretti. Le credenziali che possono chiamare le API HTTP del gateway sono a tutti gli effetti segreti di accesso completo per quel gateway.

ATTENZIONE **Conseguenza pratica, da non dimenticare mai**

Il token del gateway non si consegna a un dipendente in nessuna circostanza. Non è una password applicativa: è la chiave dell'infrastruttura di quel ruolo. Se il cliente chiede un accesso web per tutti con una password unica, la risposta è no, e la Fase 12 spiega qual è invece la soluzione corretta.

PARTE II — DECISIONI DA PRENDERE CON IL CLIENTE

7. Quanti gateway servono

È la prima domanda che fa ogni cliente. La risposta non è "uno per dipendente" e non è "uno per tutti".

REGOLA Regola di dimensionamento

Un gateway per ogni gruppo di persone che possono legittimamente vedere le stesse cose. Due persone sullo stesso gateway condividono gli stessi poteri. Se fra due persone deve esistere un segreto, servono due gateway.

Situazione	Gateway	Perché
20 dipendenti, 5 mansioni, dentro la mansione nessun segreto	5	Chi fa lo stesso lavoro vede gli stessi dati
Titolare e socio, ognuno con documenti riservati anche all'altro	2	Segreto reciproco significa confini separati
3 amministrativi, ma solo uno tratta gli stipendi	2	Gli stipendi sono un livello di fiducia a sé
Consulente esterno a tempo	1 dedicato	Si spegne e si cancella a fine incarico
Agente che legge la posta in arrivo da mittenti sconosciuti	1 dedicato	Contenuto non fidato: va isolato dagli altri ruoli

Costo in risorse. Un gateway a riposo occupa indicativamente 200-300 MB di RAM, a cui si aggiunge il container della sandbox quando l'agente lavora. Cinque ruoli stanno comodi su una macchina con 8 GB; oltre i dieci ruoli conviene salire a 16-32 GB oppure raggruppare per mansione. Misura sul campo con `systemd-cgtop` prima di promettere numeri.

Due persone nello stesso ruolo

È ammesso solo se fra loro non deve esistere alcun segreto sui dati. Anche in quel caso le loro conversazioni non devono mescolarsi: è riservatezza personale, non sicurezza dei dati, e si risolve nella Fase 8. Senza quella impostazione, per configurazione predefinita **tutti i messaggi diretti finiscono nella stessa sessione** e i due colleghi leggono la cronologia l'uno dell'altro.

8. Fase 0 — La matrice degli accessi

Da compilare con il cliente prima di toccare il server, e da far firmare. È contemporaneamente il contratto tecnico, la base della configurazione e la traccia del collaudo finale.

Ruolo	Persone	Dato o cartella	Lettura	Scrittura
Direzione	Marco	tutto	sì	sì
Amministrazione	Laura	fatture, pagamenti	sì	sì
Amministrazione	Laura	contratti dipendenti	no	no
Segreteria	Sara, Anna	anagrafica clienti (soli contatti)	sì	no
Segreteria	Sara, Anna	fatture, importi	no	no
Tutti	—	knowledge base aziendale	sì	solo Laura

REGOLA Ogni riga con un "no" diventa un test negativo obbligatorio nella Fase 16

È ciò che consegna al cliente come prova che il sistema fa quello che hai promesso, ed è ciò che giustifica il prezzo del lavoro.

La domanda che decide cosa puoi promettere

Dove stanno i dati oggi?

- **In un database.** Puoi separare fino alla singola colonna (telefono sì, importo no, sullo stesso cliente) e alla singola riga (ogni commerciale vede solo i propri clienti).
- **In cartelle, file Excel e PDF sparsi.** Puoi separare solo per cartella. Se il cliente vuole granularità fine, la migrazione dei dati su database fa parte del lavoro e va quotata come tale. Non promettere granularità di colonna su una cartella condivisa.

Le tre domande da mettere per iscritto insieme alla matrice

- **Chi è l'amministratore di sistema del server?** Quella persona vede tutto, per definizione. Va nominata nel contratto.
- **I dati possono uscire dall'azienda?** Il gateway invia il contenuto delle conversazioni al fornitore del modello. Se la risposta è no, serve un modello eseguito in locale e cambia il dimensionamento hardware.
- **Chi risponde in caso di incidente?** Definisci il canale e i tempi prima, non dopo.

9. Come entrano i dipendenti

Decisione da prendere adesso, perché determina la Fase 11 e la Fase 12.

Opzione	Come funziona	Quando sceglierla	Gestione
Bot di messaggistica dedicato	Un bot per ruolo, elenco esplicito dei mittenti ammessi, nessun gruppo	Impostazione predefinita, adatta alla grande maggioranza dei clienti	Bassa
Interfaccia web dietro reverse proxy con identità	Il proxy autentica con l'identity provider aziendale e passa l'identità verificata al gateway	Il cliente ha già Google Workspace o Microsoft Entra e vuole un'unica porta d'ingresso	Medio-alta
Interfaccia web su rete privata (VPN o tailnet)	Il gateway resta su loopback, la rete privata porta il traffico	Pochi utenti tecnici, nessun identity provider	Media
Porta del gateway esposta con token condiviso	—	Mai. Il token è accesso operatore pieno	—

REGOLA Raccomandazione

Parti dai bot di messaggistica. Aggiungi l'interfaccia web solo se il cliente la chiede esplicitamente e possiede già un identity provider aziendale: in quel caso la sicurezza del sistema diventa la sicurezza della configurazione del proxy, e questo va detto e messo per iscritto.

10. Quale modello AI

Non è una scelta di costo: è una scelta di sicurezza.

La documentazione ufficiale è netta: la resistenza all'iniezione di prompt non è uniforme fra i livelli di modello, e i modelli più piccoli ed economici sono più esposti all'uso improprio degli strumenti e al dirottamento delle istruzioni sotto prompt avversari. Per agenti che hanno strumenti in mano o che leggono contenuto non fidato, la raccomandazione ufficiale è un modello di ultima generazione e di fascia alta.

Fonte: docs.openclaw.ai/gateway/security

Come si traduce nella pratica

Profilo dell'agente	Rischio	Scelta
Riceve solo da due persone in elenco, legge una cartella in sola lettura, interroga il database	Basso	Un modello economico è accettabile
Legge la posta in arrivo, naviga il web, apre allegati	Alto	Modello di fascia alta, oppure gli si tolgono gli strumenti
Ha exec o scrittura su cartelle condivise	Alto	Modello di fascia alta e sandbox obbligatoria

- **Contenuto non fidato non significa solo mittente sconosciuto.** Anche con un solo mittente autorizzato, ogni testo che l'agente legge — pagine web, allegati, email, log incollati, documenti — può contenere istruzioni ostili rivolte al modello.
- **Se il cliente insiste su un modello economico** per un agente esposto a contenuto non fidato, riduci il raggio d'azione: solo strumenti di sola lettura, sandbox sempre attiva, nessun accesso di rete, elenchi di mittenti stretti. Mettilo nell'offerta.
- **Il dato esce dall'azienda.** Con un modello via API il contenuto delle conversazioni e dei file letti arriva al fornitore del modello. Se il cliente tratta dati particolari, è una valutazione da mettere per iscritto prima dell'installazione.

11. Cosa procurarsi dal cliente prima di iniziare

Questa è la checklist di partenza. Se manca anche una sola voce, non iniziare l'installazione: ti fermeresti a metà e dovresti tornare, e il tempo perso lo paghi tu.

Cosa ti serve	A chi lo chiedi	Senza questo non puoi
Accesso SSH con privilegi di amministratore al server, e il suo indirizzo	Referente IT	Fare nulla
Nome e cognome di chi amministra il server	Titolare	Scrivere il contratto: quella persona vede tutto
Elenco delle mansioni, con le persone di ciascuna	Titolare	Compilare la matrice della Fase 0
Identificativo di messaggistica di ogni persona	Ogni persona	Compilare a <code>lowFrom</code> nella Fase 11
Un token bot per ogni mansione	Lo crei tu con BotFather, o te lo dà il cliente	Collegare i canali
Chiave API del fornitore del modello, intestata al cliente	Titolare	Far funzionare gli agenti. Non usare la tua
Accesso ai dati: credenziali del database esistente, oppure il permesso di crearne uno	Referente IT o fornitore del gestionale	Le Fasi 4 e 10
Elenco delle cartelle da esporre, con chi le può leggere e chi scrivere	Titolare	La Fase 3
Matrice degli accessi firmata	Titolare	Consegnare: è il documento che ti protegge
Finestra di manutenzione concordata	Referente IT	Riavviare e collaudare senza fermare il lavoro

ATTENZIONE Sui token dei bot e sulle chiavi API

Intestali sempre al cliente, non a te. Se un domani il rapporto finisce, non devi ritrovarti con credenziali aziendali sul tuo account, né il cliente deve dipendere da te per rigenerarle. Consegnale insieme al verbale di collaudo.

12. La scheda d'impianto e l'albero delle decisioni

Questa guida descrive un'installazione di riferimento con tre mansioni, un database e i bot di messaggistica. Nessuna azienda è identica a quella di riferimento: quello che rende la procedura replicabile è la **scheda d'impianto**, cioè il foglio in cui scrivi una volta sola i valori di quel cliente, e da lì in poi esegui sempre gli stessi comandi.

12.1 Prima decidi la forma dell'installazione



Figura 4 — Le quattro decisioni che determinano quali fasi eseguire. Vanno prese con il cliente e riportate sulla scheda d'impianto.

12.2 Poi compili la scheda

L'Allegato A contiene la scheda vuota. Ecco come si presenta compilata:

Mansione	Utente Linux	Porta	Ruolo di database	Cartelle
Segreteria	oc-segreteria	18791	segreteria	clienti (lettura), kb (lettura)
Amministrazione	oc-amministrazione	18790	amministrazione	fatture (scrittura), clienti (lettura), kb (scrittura)
Direzione	oc-direzione	18789	direzione	tutte (scrittura), kb (lettura)

12.3 Poi esegui

Con la scheda compilata, l'installazione di ogni mansione è una sequenza sempre uguale. L'**Allegato B** contiene quella sequenza in forma parametrica: definisci quattro variabili in cima al terminale e i comandi si adattano da soli. Le fasi da 1 a 15 restano il manuale che spiega cosa fa ogni comando e cosa verificare; l'Allegato B è ciò che esegui davvero, mansione dopo mansione.

REGOLA Il metodo, in tre righe

Compila la scheda. Esegui l'Allegato B per ogni mansione. Collauda con la Fase 16. Le prime volte tieni aperte anche le fasi dettagliate; dalla terza installazione in poi ti servirà solo la scheda e l'Allegato B.

12.4 Le quattro cose che cambiano da azienda ad azienda

- **I nomi delle mansioni.** "Segreteria, amministrazione, direzione" è un esempio. In uno studio tecnico saranno "progettazione, contabilità, titolare"; in un'officina "accettazione, magazzino, titolare". Cambiano i nomi, non la struttura.
- **Il numero di mansioni.** Da una a dieci. Ogni mansione in più è la stessa sequenza ripetuta, con quattro valori diversi.
- **Dove stanno i dati.** È l'unica variabile che cambia davvero il lavoro: con un database esegui anche le Fasi 4 e 10, con sole cartelle segui l'Allegato E.
- **Il canale.** Bot di messaggistica quasi sempre; interfaccia web solo se il cliente ha già un provider di identità aziendale.

PARTE III — INSTALLAZIONE PASSO PASSO

Fase 1 — Preparazione del server e firewall

A cosa serve. Preparare la macchina che ospiterà tutti i ruoli e chiudere ogni via d'ingresso che non sia SSH. Alla fine di questa fase il server è raggiungibile solo da te e dal referente IT del cliente.

Il server deve essere **dedicato**: Linux Ubuntu LTS, sempre acceso. Mai la postazione di lavoro di un dipendente, mai una macchina usata anche per altro.

1.1 Aggiornamento e pacchetti di base

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y acl podman uidmap slirp4netns fuse-overlayfs \
    postgresql ufw curl git
```

Pacchetto	A cosa serve
acl	Permessi fini sulle cartelle, oltre al classico proprietario e gruppo (Fase 3)
podman, uidmap, fuse-overlayfs	Container senza privilegi per le sandbox: ogni utente esegue i propri, senza appartenere a gruppi speciali
postgresql	Il livello di controllo più forte: ruoli, viste e permessi per colonna (Fase 4)
ufw	Firewall

1.2 Firewall

```
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
```

Nessuna porta di OpenClaw viene aperta, né adesso né dopo: i gateway ascoltano solo sulla macchina stessa.

1.3 Irrobustimento di SSH

```
sudo sed -i 's/^#*PermitRootLogin.*/PermitRootLogin no/' /etc/ssh/sshd_config
sudo sed -i 's/^#*PasswordAuthentication.*/PasswordAuthentication no/' /etc/ssh/sshd_config
sudo systemctl restart ssh
```

ATTENZIONE Prima di eseguire questi comandi

Assicurati di avere già una chiave SSH funzionante sul server, altrimenti ti chiudi fuori. Verifica aprendo una **seconda** sessione SSH prima di chiudere la prima.

VERIFICA

```
sudo ufw status verbose mostra deny (incoming) e la sola regola OpenSSH.
ss -ltnp non mostra processi in ascolto su 0.0.0.0 oltre a SSH.
sudo -u postgres psql -c "SHOW listen_addresses;" risponde localhost.
```

NOTA Se in futuro installi Docker su questa macchina

Le porte pubblicate dai container Docker attraversano le catene di forwarding di Docker, non le regole INPUT dell'host: UFW da solo non le blocca, e le regole vanno messe nella catena DOCKER-USER. Con Podman e sandbox senza rete pubblicata il problema non si pone.

Fase 2 — Un utente Linux per ruolo

A cosa serve. Creare il confine reale fra i ruoli. Ogni gateway girerà sotto il proprio utente di sistema: il kernel non applica le istruzioni scritte nei prompt, applica i permessi di quell'utente. È anche il motivo per cui un ruolo non potrà mai leggere le conversazioni di un altro: le trascrizioni stanno nella home di ciascuno.

2.1 Creazione degli utenti

```
sudo adduser --disabled-password --gecos "" oc-direzione
sudo adduser --disabled-password --gecos "" oc-amministrazione
sudo adduser --disabled-password --gecos "" oc-segreteria
```

ATTENZIONE Due errori che annullano tutto il lavoro

1. **Mettere questi utenti nel gruppo docker.** Chi sta nel gruppo docker può diventare root sull'host: equivale a dare a ogni dipendente la chiave di tutto. È il motivo per cui le sandbox di questa installazione usano Podman senza privilegi, che non richiede alcun gruppo speciale.
2. **Dare sudo a questi utenti.** L'agente può fare tutto ciò che può fare il suo utente Linux: se l'utente ha sudo, l'agente ha sudo.

2.2 Servizio sempre attivo anche senza sessione aperta

Il gateway deve restare in esecuzione anche quando nessuno è collegato in SSH. Il linger serve a questo, e in più abilita la directory di runtime persistente di cui i container senza privilegi hanno bisogno.

```
sudo loginctl enable-linger oc-direzione
sudo loginctl enable-linger oc-amministrazione
sudo loginctl enable-linger oc-segreteria
```

2.3 Intervalli di UID subordinati

I container senza privilegi hanno bisogno di un intervallo di UID e GID subordinati per ogni utente. Su Ubuntu adduser di norma li assegna da solo: verifica, e assegnali solo se mancano.

```
grep -E 'oc-(direzione|amministrazione|segreteria)' /etc/subuid /etc/subgid

# solo se una riga manca, per esempio per oc-segreteria:
sudo usermod --add-subuids 200000-265535 --add-subgids 200000-265535 oc-segreteria
```

2.4 Come si entra in un utente di ruolo

Tutte le operazioni delle fasi successive vanno eseguite dentro l'utente del ruolo. Usa sempre questa forma, che imposta anche la directory di runtime necessaria ai container:

```
sudo -u oc-segreteria -i
export XDG_RUNTIME_DIR=/run/user/$(id -u)
# ... comandi del ruolo ...
exit
```

VERIFICA

groups oc-segreteria non contiene docker né sudo.
sudo -u oc-segreteria sudo -l risponde che l'utente non può usare sudo.
loginctl show-user oc-segreteria | grep Linger risponde Linger=yes.
Dentro l'utente, podman info risponde senza errori e mostra rootless: true.

Fase 3 — Cartelle dei dati, permessi e ACL

A cosa serve. Costruire il livello 2: il permesso di lettura e scrittura sulle cartelle aziendali, imposto dal kernel. Va completata e verificata prima di installare OpenClaw.

3.1 Struttura

```
sudo mkdir -p /srv/dati/fatture /srv/dati/clienti /srv/dati/contratti /srv/kb
sudo chown root:root /srv /srv/dati
sudo chmod 755 /srv /srv/dati
sudo chown root:root /srv/dati/* /srv/kb
sudo chmod 750 /srv/dati/* /srv/kb
```

/srv e /srv/dati restano attraversabili da tutti ma non leggibili nel contenuto delle sottocartelle: i permessi veri stanno sulle cartelle foglia, che di base non sono accessibili a nessuno se non a root.

3.2 Permessi assegnati per utente, non per gruppo

Il controllo viene dato con ACL POSIX **a nome dell'utente del ruolo**. Il motivo è tecnico e va capito, perché è il tipo di dettaglio che fa fallire l'installazione in modo silenzioso: dentro un container senza privilegi il processo mantiene l'identità dell'utente che lo ha avviato, mentre i gruppi supplementari non vengono mappati per configurazione predefinita. Un permesso concesso al gruppo può quindi non essere visto dalla sandbox; un permesso concesso all'utente sì.

```
# fatture: amministrazione scrive, direzione legge
sudo setfacl -m u:oc-amministrazione:rwX /srv/dati/fatture
sudo setfacl -m u:oc-direzione:rx /srv/dati/fatture
sudo setfacl -d -m u:oc-amministrazione:rw- /srv/dati/fatture
sudo setfacl -d -m u:oc-direzione:r-- /srv/dati/fatture

# clienti: segreteria legge, amministrazione e direzione scrivono
sudo setfacl -m u:oc-segreteria:rx /srv/dati/clienti
sudo setfacl -m u:oc-amministrazione:rwX /srv/dati/clienti
sudo setfacl -m u:oc-direzione:rwX /srv/dati/clienti
sudo setfacl -d -m u:oc-segreteria:r-- /srv/dati/clienti
sudo setfacl -d -m u:oc-amministrazione:rw- /srv/dati/clienti
sudo setfacl -d -m u:oc-direzione:rw- /srv/dati/clienti

# contratti: solo direzione
sudo setfacl -m u:oc-direzione:rwX /srv/dati/contratti
sudo setfacl -d -m u:oc-direzione:rw- /srv/dati/contratti
```

Elemento	Significato
-m	Aggiunge una voce di permesso valida da subito sulla cartella
-d	Permessi predefiniti: valgono per i file che verranno creati in futuro dentro quella cartella
u:nome:rx	All'utente indicato: lettura ed elenco del contenuto, nessuna scrittura
u:nome:rwX	All'utente indicato: lettura, scrittura ed elenco del contenuto

Senza le righe -d, un file creato domani eredita solo i permessi classici e il ruolo che dovrebbe leggerlo resta fuori. È la causa più comune di "funzionava, poi ha smesso".

VERIFICA

```
getfacl /srv/dati/fatture elenca user:oc-amministrazione:rwX e user:oc-direzione:r-x.
sudo -u oc-segreteria ls /srv/dati/fatture risponde Permission denied.
sudo -u oc-segreteria ls /srv/dati/clienti elenca invece il contenuto.
Se il primo comando elenca i file, fermati qui: tutto il resto poggia su questa verifica.
```

Fase 4 — PostgreSQL: ruoli, viste e riservatezza per colonna

A cosa serve. Costruire il livello 1, l'unico che può separare i dati fino alla singola colonna e alla singola riga. È anche la risposta tecnica alla domanda "e se l'intelligenza artificiale si inventa un dato?".

4.1 Database e chiusura predefinita

```
sudo -u postgres createdb azienda
sudo -u postgres psql -d azienda
```

```
REVOKE CREATE ON SCHEMA public FROM PUBLIC;
REVOKE ALL ON ALL TABLES IN SCHEMA public FROM PUBLIC;
ALTER DEFAULT PRIVILEGES IN SCHEMA public REVOKE ALL ON TABLES FROM PUBLIC;
```

Si parte da tutto chiuso e si aprono solo i permessi necessari, uno per uno. La terza riga vale anche per le tabelle che verranno create in futuro.

4.2 Un ruolo di database per ogni ruolo aziendale

```
CREATE ROLE segreteria      LOGIN PASSWORD 'password-lunga-e-casuale-1';
CREATE ROLE amministrazione LOGIN PASSWORD 'password-lunga-e-casuale-2';
CREATE ROLE direzione       LOGIN PASSWORD 'password-lunga-e-casuale-3';
```

Genera le password con `openssl rand -base64 24`. Non riusarle fra ruoli e non scriverle in nessun file di configurazione di OpenClaw: la Fase 10 mostra dove vanno.

4.3 Le viste: si espongono le colonne, non le tabelle

```
CREATE VIEW clienti_contatti AS
  SELECT id, ragione_sociale, telefono, email, citta
  FROM clienti;

GRANT USAGE ON SCHEMA public TO segreteria, amministrazione, direzione;
GRANT SELECT ON clienti_contatti TO segreteria, amministrazione, direzione;
```

Perché funziona. In PostgreSQL una vista viene eseguita con i privilegi del proprietario. L'utente `segreteria` legge la vista pur non avendo alcun diritto sulla tabella `clienti`. Vede le colonne che hai messo nella vista e nient'altro: le colonne economiche, per quel ruolo, non esistono proprio. Non sono nascoste: non sono raggiungibili.

4.4 Amministrazione: scrittura senza riscrittura della storia

```
GRANT SELECT, INSERT ON fatture TO amministrazione;
GRANT USAGE, SELECT ON ALL SEQUENCES IN SCHEMA public TO amministrazione;
-- niente UPDATE/DELETE: le correzioni si fanno con note di credito,
-- non riscrivendo la contabilità'
```

4.5 Quando serve "vede solo i suoi clienti"

```
ALTER TABLE clienti ENABLE ROW LEVEL SECURITY;
ALTER TABLE clienti FORCE ROW LEVEL SECURITY;

CREATE POLICY clienti_propri ON clienti
  FOR SELECT TO commerciali
  USING (referente = current_user);
```

ENABLE attiva il filtro per riga; FORCE lo applica anche al proprietario della tabella. Senza FORCE la politica sembra attiva ma per il proprietario non lo è, ed è un errore che in collaudo non si vede se provi solo con i ruoli operativi.

4.6 Il determinismo delle risposte

Il cliente chiederà se l'assistente può inventarsi un dato. La risposta corretta è architetturale, non un prompt migliore:

- L'agente non ricorda i dati: esegue una query e riporta l'output.
- Se la query restituisce zero righe, la risposta è "nessun risultato".
- Se il ruolo non ha il permesso, il database restituisce un errore e l'agente lo riporta.

Nel file di comportamento del ruolo si scrive comunque la regola ("rispondi solo con i dati restituiti dallo strumento; zero righe significa nessun risultato; non stimare, non dedurre"), ma la garanzia è il GRANT mancante, non la frase.

VERIFICA

```
sudo -u oc-segreteria psql -h 127.0.0.1 -U segreteria -d azienda -c "SELECT * FROM fatture  
LIMIT 1;" risponde permission denied for table fatture. Non "0 righe": permission denied.
```

La stessa query su `clienti_contatti` funziona.

```
SELECT * FROM clienti;
```

con l'utente `segreteria` fallisce: è accessibile la vista, non la tabella.

Fase 5 — L'immagine della sandbox

A cosa serve. Preparare il container dentro cui verranno eseguiti gli strumenti dell'agente. È una fase breve ma obbligatoria: OpenClaw non sostituisce l'immagine con una generica se manca, e tutte le esecuzioni in sandbox falliscono subito con l'istruzione di build. Con i container senza privilegi **le immagini sono per utente**, quindi la costruzione va ripetuta dentro ogni ruolo.

5.1 Costruzione dell'immagine (da ripetere per ogni ruolo)

```
sudo -u oc-segreteria -i
export XDG_RUNTIME_DIR=/run/user/$(id -u)

podman build -t openclaw-sandbox:bookworm-slim - <<'DOCKERFILE'
FROM debian:bookworm-slim
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    bash ca-certificates curl git jq python3 ripgrep \
    && rm -rf /var/lib/apt/lists/*
RUN useradd --create-home --shell /bin/bash sandbox
USER sandbox
WORKDIR /home/sandbox
CMD ["sleep", "infinity"]
DOCKERFILE

exit
```

NOTA Perché proprio questa immagine

È l'immagine di riferimento della documentazione ufficiale. Contiene python3, che serve agli helper di scrittura e modifica dei file dentro la sandbox: un'immagine più povera fa fallire quegli strumenti in modo poco leggibile. Non contiene Node: se un giorno una skill lo richiede, si costruisce un'immagine dedicata e non si installa nulla a runtime, perché la sandbox gira senza rete e con il filesystem di root in sola lettura.

5.2 Annota UID e GID di ciascun ruolo

Servono nella Fase 9: nei container senza privilegi il processo mantiene l'identità dell'utente dell'host, e va dichiarata in configurazione.

```
id -u oc-segreteria
id -g oc-segreteria
```

VERIFICA

Dentro ciascun utente di ruolo, `podman images` elenca `openclaw-sandbox:bookworm-slim`.
`podman run --rm openclaw-sandbox:bookworm-slim python3 --version` stampa una versione di Python.

Fase 6 — Installazione del gateway

A cosa serve. Installare OpenClaw dentro l'utente del ruolo e registrarlo come servizio, in modo che riparta da solo al riavvio del server.

Da qui in avanti tutte le fasi vanno ripetute **identiche per ogni ruolo**, dentro l'utente corrispondente. Gli esempi usano il ruolo Segreteria, che è il più restrittivo: gli altri due si ottengono cambiando i valori indicati nell'Allegato D.

6.1 Requisiti di runtime

OpenClaw richiede Node 22.22.3 o superiore, 24.15 o superiore, oppure 25.9 o superiore; Node 26 è il default consigliato e l'installer lo installa da solo se manca.

6.2 Installazione

```
sudo -u oc-segreteria -i
export XDG_RUNTIME_DIR=/run/user/$(id -u)

curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard --install-daemon
```

L'onboarding installa il gateway come servizio utente systemd (per questo serviva il linger della Fase 2) e guida nella scelta del fornitore del modello, della chiave API e della cartella di lavoro. **Alla domanda sui canali, salta la configurazione:** i canali si collegano nella Fase 11, dopo aver messo in sicurezza gli strumenti. Un agente raggiungibile prima di essere configurato è un agente esposto.

6.3 Verifica dell'installazione

```
openclaw --version
openclaw doctor
openclaw gateway status
exit
```

Comando	Cosa fa
openclaw --version	Mostra la versione installata: annotata, serve in caso di segnalazione
openclaw doctor	Controlla la coerenza della configurazione e propone correzioni sicure
openclaw gateway status	Dice se il servizio è attivo e risponde

VERIFICA

openclaw gateway status riporta il servizio in esecuzione.
openclaw doctor non riporta errori bloccanti; gli avvisi di configurazione incompleta sono normali a questo punto.
Se compare un errore di stato già in uso, hai eseguito il comando dentro l'utente sbagliato: controlla con whoami.

Fase 7 — Configurazione del gateway

A cosa serve. Decidere su quale porta ascolta il gateway, da dove è raggiungibile e con quale credenziale. Il file da modificare è `~/openclaw/openclaw.json` dell'utente del ruolo. Le fasi da 7 a 14 costruiscono quel file un blocco alla volta; l'Allegato C contiene il risultato completo.

7.1 Generazione del token

```
openclaw doctor --generate-gateway-token
```

Il token serve a te e agli strumenti locali di amministrazione, non ai dipendenti.

7.2 Blocco gateway

```
{
  "gateway": {
    "mode": "local",
    "bind": "loopback",
    "port": 18791,
    "auth": { "mode": "token", "token": "TOKEN-SOLO-DI-QUESTO-GATEWAY" }
  }
}
```

7.3 Che cosa significano queste impostazioni

Chiave	Valore	Effetto e conseguenza se sbagliata
mode	local	Installazione locale, senza collegamento a un gateway remoto
bind	loopback	Ascolta solo sulla macchina stessa. I valori <code>lan</code> , <code>tailnet</code> e <code>custom</code> allargano la superficie di attacco e richiedono firewall e autenticazione reali
port	una per ruolo	La predefinita è 18789: se non la cambi sugli altri ruoli, i gateway si scontrano e uno non parte
auth.mode	token	L'autenticazione è obbligatoria: senza una credenziale valida il gateway rifiuta le connessioni
auth.token	diverso per ruolo	Chi possiede il token ha accesso operatore pieno a quel gateway. Mai copiarlo da un ruolo all'altro, mai consegnarlo a un dipendente

Ruolo	Utente	Porta	File di configurazione
Direzione	oc-direzione	18789	/home/oc-direzione/openclaw/openclaw.json
Amministrazione	oc-amministrazione	18790	/home/oc-amministrazione/openclaw/openclaw.json
Segreteria	oc-segreteria	18791	/home/oc-segreteria/openclaw/openclaw.json

Sulla stessa porta il gateway espone anche l'interfaccia di controllo: esporla in rete equivale a consegnare il controllo dell'agente. È la ragione per cui `bind` resta su `loopback` salvo il caso della Fase 12.

VERIFICA

`ss -ltnp | grep 1879` mostra le porte in ascolto **solo** su `127.0.0.1`.
Da un'altra macchina, `nc -vz INDIRIZZO-DEL-SERVER 18791` fallisce.

Fase 8 — Agente, tool policy e isolamento delle sessioni

A cosa serve. Decidere che cosa l'agente ha in mano e impedire che due persone dello stesso ruolo si leggano a vicenda. Tre blocchi, tutti obbligatori.

8.1 Isolamento delle sessioni

Per configurazione predefinita OpenClaw instrada **tutti i messaggi diretti nella stessa sessione**, perché il caso previsto è quello di una sola persona che scrive da più dispositivi. In un'installazione dove più persone scrivono allo stesso gateway questo significa che i colleghi leggono la conversazione l'uno dell'altro.

```
{
  "session": { "dmScope": "per-channel-peer" },
  "tools": { "sessions": { "visibility": "self" } }
}
```

Valore di dmScope	Effetto
main (predefinito)	Tutti i messaggi diretti condividono una sola sessione
per-channel-peer	Ogni coppia canale più mittente ha un contesto isolato: è la modalità corretta per una casella condivisa
per-account-channel-peer	Come sopra, separato anche per account del canale

tools.sessions.visibility decide quali sessioni gli strumenti di sessione possono raggiungere: `self` limita a quella corrente. Il valore predefinito è `tree`, che include anche le sessioni di gruppo osservate in modo ambientale; il valore `all` permette a un agente di leggere la cronologia di tutti gli altri e non va mai usato in azienda.

Fonti: docs.openclaw.ai/gateway/security e [/gateway/config-tools](https://docs.openclaw.ai/gateway/config-tools)

8.2 Politiche globali degli strumenti

```
{
  "tools": {
    "elevated": { "enabled": false },
    "exec": { "security": "deny", "ask": "always" },
    "toolsBySender": {
      "": { "deny": ["group:runtime", "write", "edit", "apply_patch"] }
    }
  }
}
```

Chiave	Effetto
elevated.enabled: false	Disattiva la via di fuga che esegue comandi fuori dalla sandbox. Va disattivata esplicitamente
exec.security: "deny"	Nega l'esecuzione di comandi anche quando arriva per altre strade
toolsBySender	Rete di sicurezza valutata sul mittente del turno corrente. È difesa in profondità: non autentica il resto del contenuto del prompt, quindi non sostituisce i livelli 1, 2 e 3

8.3 L'agente del ruolo

```
{
  "agents": {
    "entries": {
      "segreteria": {
        "default": true,
        "name": "Assistente Segreteria",
        "workspace": "/home/oc-segreteria/.openclaw/workspace",
        "tools": {
          "allow": ["read", "bundle-mcp"],
          "deny": [
            "exec", "process", "code_execution",
            "write", "edit", "apply_patch",
            "browser", "canvas", "screen", "terminal",
            "gateway", "cron", "nodes", "computer",
            "group:sessions", "web_search", "web_fetch"
          ]
        }
      }
    }
  }
}
```

Elemento	Significato
agents.entries	Mappa degli agenti, con l'identificativo dell'agente come chiave
default: true	Esattamente un agente per gateway deve averlo: è quello che risponde quando nessun binding dice altro
workspace	Cartella di lavoro dell'agente, dove stanno i suoi file di istruzioni
tools.allow	L'elenco di ciò che esiste per questo agente. bundle-mcp abilita gli strumenti dei server MCP configurati (Fase 10)
tools.deny	Il divieto vince sempre sul permesso, a ogni livello della catena

ATTENZIONE Tre trappole da conoscere

- 1. **write e apply_patch sono strumenti distinti.** Negare write non nega apply_patch: vanno elencati entrambi, insieme a edit.
- 2. **Non negare group:fs se all'agente serve read:** quel gruppo include anche la lettura, e il divieto vince.
- 3. **Negare la scrittura lasciando exec non serve a nulla.** La documentazione lo dice apertamente: quella politica disabilita gli strumenti di filesystem, ma exec resta una shell e scrive dove il filesystem glielo consente. Per un agente in sola lettura vanno negati anche exec e process.

Strumento	Direzione	Ruoli operativi	Motivo
read	sì	sì	Serve per leggere i documenti consentiti
MCP del database	sì	sì	Unico accesso ai dati strutturati, con le credenziali del ruolo
write, edit, apply_patch	sì	solo se previsto	Concedere solo se il ruolo deve davvero produrre file
exec, process	con cautela	no	È una shell: aggira ogni restrizione applicativa sui file
browser	opzionale	no	Superficie di attacco e via di esfiltrazione
gateway, cron	no	no	Strumenti del piano di controllo
group:sessions	no	no	Lettura e creazione di sessioni, incluse quelle altrui

VERIFICA

Dopo il riavvio del gateway, chiedi all'agente dal suo canale di eseguire il comando `ls /.` . Deve rispondere che lo strumento non è disponibile, non "non sono autorizzato".

Fase 9 — Sandbox e cartelle montate

A cosa serve. Spostare l'esecuzione degli strumenti dentro un container che vede solo le cartelle che gli monti. Il processo del gateway resta sull'host; ciò che si sposta è l'esecuzione. È il confine che rende reale la separazione delle cartelle e che tiene fuori dalla portata dell'agente la sua stessa configurazione.

9.1 Blocco di configurazione

Sostituisci 1002:1002 con l'UID e il GID reali dell'utente del ruolo, letti nella Fase 5.2.

```
{
  "agents": {
    "defaults": {
      "sandbox": {
        "mode": "all",
        "backend": "podman",
        "scope": "agent",
        "workspaceAccess": "ro",
        "docker": {
          "image": "openclaw-sandbox:bookworm-slim",
          "user": "1002:1002",
          "network": "none",
          "readOnlyRoot": true,
          "capDrop": ["ALL"],
          "tmpfs": ["/tmp", "/var/tmp", "/run"],
          "pidsLimit": 256,
          "memory": "1g",
          "cpus": 1,
          "dangerouslyAllowExternalBindSources": true,
          "binds": [
            "/srv/dati/clienti:/dati/clienti:ro",
            "/srv/kb:/kb:ro"
          ]
        }
      }
    }
  }
}
```

9.2 Che cosa significano queste impostazioni

Chiave	Valore	Effetto
mode	all	Tutti gli strumenti girano nel container. Il valore predefinito è off: senza questa riga non c'è sandbox
backend	podman	Container senza privilegi, eseguiti dall'utente del ruolo
scope	agent	Un container per agente. shared ignora gli override per agente: da evitare qui
workspaceAccess	ro	La cartella di lavoro è montata in sola lettura su /agent
user	UID:GID del ruolo	Nei container senza privilegi il processo mantiene l'identità dell'utente dell'host: è ciò che fa combaciare le ACL della Fase 3
network	none	Nessuna rete in uscita dal container. È anche il valore predefinito
readOnlyRoot	true	Filesystem di root del container non scrivibile
capDrop	["ALL"]	Nessuna capability Linux concessa al container
tmpfs	tre percorsi	Le uniche aree scrivibili, in memoria, che spariscono alla chiusura
pidsLimit, memory, cpus	limiti	Contengono i danni di un processo impazzito

Chiave	Valore	Effetto
<code>binds</code>	<code>host:container</code> <code>:ro</code> oppure <code>:rw</code>	Le uniche cartelle esterne visibili all'agente

9.3 Perché i bind mount sono il confine giusto

Il container costruisce la propria vista del filesystem con isolamento dei mount, e la modalità `ro` o `rw` vale per **ogni processo** dentro la sandbox: copre i comandi, gli strumenti di file, i processi figli e le librerie, senza dover ripetere il controllo in ogni percorso di codice. Un elenco di percorsi consentiti a livello applicativo non offre lo stesso confine, perché una shell o una libreria può accedere ai file direttamente.

9.4 Quattro avvertenze operative

- `dangerouslyAllowExternalBindSources` serve perché `/srv/...` sta fuori dalle radici della cartella di lavoro. Va attivato consapevolmente, montando solo il minimo indispensabile. Non disabilita gli altri controlli: restano bloccati i percorsi di sistema, le credenziali nella home (`.ssh`, `.aws`, `.config`, `.gnupg`, `.npm`), il socket del motore dei container e le fughe tramite collegamenti simbolici sui percorsi padre.
- **Il socket del motore dei container non va mai montato dentro la sandbox:** equivale a dare root sull'host.
- **Il doppio lucchetto è voluto.** Anche se per errore scrivi `:rw` su una cartella che il ruolo può solo leggere, l'ACL impostata nella Fase 3 blocca comunque la scrittura.
- **Non impostare `tools.fs.workspaceOnly` in questa configurazione:** limiterebbe gli strumenti di file alla sola cartella di lavoro e renderebbe invisibili le cartelle montate. Qui il confine è già dato dalla sandbox, che non monta nient'altro.

9.5 Applicazione delle modifiche

```
openclaw sandbox recreate --agent segreteria
openclaw gateway restart
openclaw sandbox explain --agent segreteria
```

Il container non recepisce le modifiche da solo: va ricreato. `sandbox explain` mostra la configurazione effettiva, cioè quella che il sistema sta davvero applicando, ed è il comando da usare ogni volta che qualcosa non torna.

VERIFICA

```
openclaw sandbox explain --agent segreteria
```

mostra modalità `all`, backend Podman, cartella di lavoro in sola lettura e **solo** i mount dichiarati.

```
podman ps
```

dentro l'utente del ruolo mostra un container attivo quando l'agente lavora.

Dal canale: leggere un file in `/dati/clienti` riesce; leggere `/etc/passwd` o `/srv/dati/fatture` fallisce.

Fase 10 — Accesso al database tramite MCP

A cosa serve. Dare all'agente un modo per interrogare il database senza aprire alcuna rete alla sandbox. Il punto tecnico da capire è questo: gli strumenti forniti da plugin e da server MCP **vengono eseguiti lato gateway**, cioè sull'host come utente del ruolo, non dentro il container. Il container può quindi restare completamente senza rete, e il database viene raggiunto su 127.0.0.1 con le credenziali del ruolo.

10.1 Credenziali del ruolo fuori dalla configurazione

La password del database non va scritta in `openclaw.json`. Si usa il file standard di PostgreSQL nella home del ruolo, leggibile solo da quell'utente:

```
sudo -u oc-segreteria -i
printf '127.0.0.1:5432:azienda:segreteria:PASSWORD-DEL-RUOLO\n' > ~/.pgpass
chmod 600 ~/.pgpass
psql -h 127.0.0.1 -U segreteria -d azienda -c "SELECT 1;"
exit
```

10.2 Dichiarazione del server MCP

```
{
  "mcp": {
    "servers": {
      "db": {
        "command": "npx",
        "args": ["-y", "PACCHETTO-MCP-POSTGRES@VERSIONE-ESATTA"],
        "env": {
          "PGHOST": "127.0.0.1",
          "PGPORT": "5432",
          "PGDATABASE": "azienda",
          "PGUSER": "segreteria"
        }
      }
    }
  },
  "tools": {
    "sandbox": { "tools": { "alsoAllow": ["bundle-mcp"] } }
  }
}
```

ATTENZIONE Due cose da non sbagliare

1. `tools.sandbox.tools.alsoAllow` con `bundle-mcp` è **obbligatorio**. Con la sandbox attiva, gli strumenti MCP vengono filtrati prima della chiamata al modello anche se il server si carica correttamente: il sintomo è un agente che dice di non avere lo strumento, senza errori nei log. `openclaw doctor` intercetta questa situazione.
2. **Fissa la versione esatta del pacchetto e verificane la provenienza.** I plugin e i server MCP girano con i privilegi del gateway, cioè come utente del ruolo e con accesso alle credenziali di quel ruolo. Installa solo da fonti verificate e preferisci versioni bloccate.

VERIFICA

`openclaw doctor` non segnala strumenti MCP filtrati dalla policy della sandbox.
Dal canale del ruolo, una domanda sui contatti dei clienti restituisce righe reali.
Una domanda sulle fatture restituisce l'errore di permesso del database, non una risposta inventata né un rifiuto gentile.

Fase 11 — Canali di messaggistica e instradamento delle persone

A cosa serve. Collegare le persone al loro gateway e chiudere la porta a tutti gli altri. Un bot dedicato per ogni gateway: il bot è la porta d'ingresso di quel confine di fiducia e non viene mai condiviso.

11.1 Token del bot fuori dalla configurazione

```
sudo -u oc-segreteria -i
printf '123456:AA...il-token-del-bot\n' > ~/.openclaw/telegram.token
chmod 600 ~/.openclaw/telegram.token
exit
```

Il file del token deve essere un file regolare: i collegamenti simbolici vengono rifiutati.

11.2 Blocco canali

```
{
  "channels": {
    "telegram": {
      "tokenFile": "/home/oc-segreteria/.openclaw/telegram.token",
      "dmPolicy": "allowlist",
      "allowFrom": ["ID_TELEGRAM_DI_SARA", "ID_TELEGRAM_DI_ANNA"],
      "groups": {},
      "contextVisibility": "allowlist"
    }
  }
}
```

Impostazione	Perché così
dmPolicy: "allowlist"	I mittenti sconosciuti vengono bloccati senza nemmeno la procedura di abbinamento. Le alternative sono pairing (predefinita, con codice), open e disabled
allowFrom	Un identificativo per persona, presi dalla matrice della Fase 0. Mai "*"
groups: {}	Nessun gruppo ammesso. In un gruppo chiunque può scrivere all'agente e ne eredita i poteri
contextVisibility: "allowlist"	Il contesto supplementare (citazioni, testo inoltrato, cronologia) viene filtrato ai soli mittenti autorizzati

ATTENZIONE Non lasciare mai allowFrom vuoto

Se l'elenco di un canale è vuoto oppure contiene "*", i comandi risultano di fatto aperti per quel canale. L'elenco vuoto non significa "nessuno": significa "nessun filtro".

11.3 Se il ruolo ha più agenti dentro lo stesso gateway

Solo in questo caso servono i binding espliciti, cioè le regole che dicono quale agente risponde a quale conversazione. Il campo si chiama channel e il tipo di destinatario è direct.

```
{
  "bindings": [
    {
      "agentId": "segreteria",
      "match": {
        "channel": "telegram",
        "peer": { "kind": "direct", "id": "ID_TELEGRAM_DI_SARA" }
      }
    }
  ]
}
```

REGOLA Limite da dichiarare al cliente

L'identità sul canale di messaggistica è un instradamento, non un login. Se un dipendente presta il telefono sbloccato, l'agente risponde a chi ha il telefono. La protezione che regge in quel caso è quella del database e dei permessi, non quella del canale.

VERIFICA

Un messaggio da un account non presente in `allowFrom` non riceve alcuna risposta.

Un messaggio da un account autorizzato riceve risposta.

Il bot invitato in un gruppo non risponde ai messaggi del gruppo.

Fase 12 — Accesso da interfaccia web (opzionale)

A cosa serve. Dare ai dipendenti una pagina web al posto del bot. Da attivare **solo** se il cliente lo ha chiesto esplicitamente e possiede già un provider di identità aziendale. Altrimenti salta questa fase: i bot della Fase 11 bastano e hanno una superficie di attacco molto minore.

12.1 Il principio

Dentro OpenClaw non esiste un "login dipendente". L'autenticazione con token o password è accesso operatore pieno a quel gateway. L'unico modo corretto è delegare l'autenticazione a un reverse proxy che conosce le identità aziendali (OAuth, OIDC, SAML) e che passa l'identità verificata al gateway in un'intestazione HTTP.

12.2 Dove deve stare il proxy

È la decisione che fa la differenza fra una configurazione sicura e una apparentemente sicura. Prima di guardare le intestazioni, il gateway applica tre controlli in quest'ordine:

- L'indirizzo di origine della richiesta deve essere elencato in `gateway.trustedProxies`, altrimenti la richiesta è rifiutata.
- Le richieste che arrivano da loopback sono rifiutate, a meno di abilitare esplicitamente `allowLoopback`.
- Le richieste che arrivano da un indirizzo coincidente con una delle interfacce di rete dell'host sono rifiutate come tentativo di spoofing.

REGOLA Conseguenza pratica

Il reverse proxy deve stare su una **macchina diversa** dal gateway, sulla rete privata. Un proxy sullo stesso host ricade nel primo o nel terzo controllo, e per farlo funzionare dovresti abilitare `allowLoopback`: in un'installazione con più ruoli questo è pericoloso, perché ogni utente locale può raggiungere la porta di loopback degli altri ruoli e presentare intestazioni di identità arbitrarie. Se non puoi mettere il proxy su un'altra macchina, non usare questa modalità: usa una rete privata e i bot della Fase 11.

12.3 Configurazione del gateway

```
{
  "gateway": {
    "bind": "lan",
    "port": 18791,
    "trustedProxies": ["10.0.0.7"],
    "controlUi": { "allowedOrigins": ["https://segreteria.azienda.it"] },
    "auth": {
      "mode": "trusted-proxy",
      "identityScopes": {
        "sara@azienda.it": ["operator.read", "operator.write"],
        "anna@azienda.it": ["operator.read", "operator.write"]
      },
      "trustedProxy": {
        "userHeader": "x-auth-request-email",
        "requiredHeaders": ["x-forwarded-proto", "x-forwarded-host"],
        "allowUsers": ["sara@azienda.it", "anna@azienda.it"],
        "allowLoopback": false,
        "deviceAutoApprove": { "enabled": false }
      }
    }
  }
}
```

ATTENZIONE Il token va rimosso

Il gateway rifiuta l'avvio se sono configurati insieme la modalità `trusted-proxy` e un token condiviso: sono mutuamente esclusivi, perché il token permetterebbe di autenticarsi su un percorso diverso da quello verificato dal proxy.

12.4 Gli scope: cosa dare e cosa non dare mai

Scope	Significato
<code>operator.read</code>	Sola lettura: stato, elenchi, cronologia delle sessioni
<code>operator.write</code>	Azioni che modificano: inviare messaggi, invocare strumenti
<code>operator.admin</code>	Amministrazione: modifica della configurazione, aggiornamenti, approvazioni ad alto rischio. Non va mai assegnato a un dipendente

- `deviceAutoApprove` **disabilitato**. Se lo attivi elencando `operator.admin`, ogni utente autenticato dal proxy ottiene automaticamente un dispositivo con privilegi di amministrazione: il controllo di sicurezza segnala la condizione come critica e il gateway registra un avviso all'avvio.
- `allowUsers` **sempre valorizzato**. Un elenco vuoto significa "chiunque sia autenticato dal proxy".
- `trustedProxies` **minimale**. Solo gli indirizzi reali del proxy, non interi sottoreti.
- **Il proxy deve sovrascrivere** le intestazioni `x-forwarded-*` ricevute dal client, non accodarle.
- **TLS terminato sul proxy** e `controlUi.allowedOrigins` esplicito: l'interfaccia di controllo richiede un contesto sicuro per generare l'identità del dispositivo.

NOTA Il controllo di sicurezza segnalerà questa modalità come critica

È voluto: ricorda che stai delegando la sicurezza alla configurazione del tuo proxy. Documenta la scelta nella consegna al cliente invece di provare a farla sparire.

L'Allegato F contiene le configurazioni pronte per Caddy e per nginx con oauth2-proxy.

Fase 13 — Knowledge base condivisa

A cosa serve. Dare a tutti i ruoli una memoria aziendale comune: procedure, listini, modelli di documento. Alcuni la possono solo leggere, altri modificare.

ATTENZIONE La memoria interna dell'agente non è una knowledge base

La memoria di OpenClaw (cartella di lavoro, trascrizioni, memoria di sessione) è per agente e viene potata automaticamente secondo le politiche di manutenzione delle sessioni. La fonte di verità condivisa deve stare fuori: in `/srv/kb` o nel database.

13.1 Se la knowledge base è fatta di documenti

```
sudo setfacl -m u:oc-segreteria:rx      /srv/kb
sudo setfacl -m u:oc-direzione:rx      /srv/kb
sudo setfacl -m u:oc-amministrazione:rw /srv/kb
sudo setfacl -d -m u:oc-segreteria:r-- /srv/kb
sudo setfacl -d -m u:oc-direzione:r--  /srv/kb
sudo setfacl -d -m u:oc-amministrazione:rw- /srv/kb
```

Poi il montaggio nella sandbox, diverso per ruolo:

```
// gateway segreteria e direzione
"binds": ["/srv/kb:/kb:ro"]

// gateway amministrazione
"binds": ["/srv/kb:/kb:rw"]
```

I due controlli si sommano: la modalità del mount e l'ACL sull'host devono essere entrambe permissive perché una scrittura riesca.

13.2 Tracciabilità delle modifiche

```
sudo git -C /srv/kb init
sudo git -C /srv/kb config user.name "kb"
sudo git -C /srv/kb config user.email "kb@azienda.local"
sudo git -C /srv/kb add -A && sudo git -C /srv/kb commit -m "baseline"
```

Con un commit periodico sai sempre chi ha cambiato cosa e torni indietro con un comando. È la differenza fra una cartella condivisa e una fonte di verità.

13.3 Se la knowledge base è fatta di dati strutturati

```
GRANT SELECT ON kb_articoli TO segreteria, direzione;
GRANT SELECT, INSERT, UPDATE ON kb_articoli TO amministrazione;
```

Preferibile quando servono ricerche, filtri e risposte deterministiche.

Fase 14 — Segreti e permessi dei file

A cosa serve. Assicurarsi che ogni gateway possieda solo le credenziali del proprio ruolo e che nessun altro utente della macchina possa leggerle.

14.1 Permessi

```
sudo chmod 700 /home/oc-segreteria/.openclaw
sudo chmod 600 /home/oc-segreteria/.openclaw/openclaw.json
sudo chmod 600 /home/oc-segreteria/.openclaw/telegram.token
sudo chmod 600 /home/oc-segreteria/.pgpass
```

Il comando `openclaw security audit --fix` applica correzioni conservative nella stessa direzione: converte le politiche di gruppo aperte in elenchi espliciti e stringe i permessi di stato, configurazione e file inclusi, portando i file a 600 e le cartelle a 700.

14.2 Cosa contiene la cartella di stato

Percorso	Contenuto
<code>openclaw.json</code>	Token del gateway, impostazioni del fornitore, elenchi di autorizzazione
<code>credentials/</code>	Credenziali dei canali ed elenchi di abbinamento
<code>agents/<id>/agent/*.sqlite</code>	Stato per agente, profili di autenticazione dei modelli, sessioni e trascrizioni
<code>sandboxes/</code>	Cartelle di lavoro delle sandbox: possono accumulare copie dei file letti e scritti

Le trascrizioni contengono i messaggi e l'output degli strumenti. Con un utente di sistema per ruolo, un ruolo non può leggere le trascrizioni di un altro: è uno dei motivi principali di questa architettura.

14.3 Rotazione

- Ogni sei mesi, e a ogni uscita di una persona dal ruolo.
- Cosa ruotare: token del gateway, password del ruolo nel database, token del bot, chiavi API del modello.
- Dopo ogni rotazione: `openclaw gateway restart` e verifica che le vecchie credenziali non funzionino più.

Fase 15 — Registro append-only

A cosa serve. Avere un registro delle attività che nessuno possa modificare o cancellare, nemmeno chi lo scrive.

15.1 Il file

```
sudo mkdir -p /var/log/openclaw-audit
sudo touch /var/log/openclaw-audit/attivita.log
sudo groupadd -f oc-audit
sudo chown root:oc-audit /var/log/openclaw-audit/attivita.log
sudo chmod 640 /var/log/openclaw-audit/attivita.log
sudo chattr +a /var/log/openclaw-audit/attivita.log
sudo usermod -aG oc-audit oc-direzione
```

Con l'attributo `+a` il file si può solo allungare. Per la rotazione serve `root`: togliere l'attributo, ruotare, rimetterlo. Funziona nativamente su ext4.

15.2 Chi scrive, e cosa

Sii preciso con il cliente su questo punto, perché è quello dove si promette più di quanto si mantenga. Le fonti di verità sono tre, e nessuna delle tre è scritta dall'agente:

- **I log del gateway**, uno per ruolo, prodotti dal processo del gateway sull'host. Si consultano con `openclaw logs` dentro l'utente del ruolo.
- **Le trascrizioni delle sessioni**, nella cartella di stato di quel ruolo.
- **La cronologia git** della knowledge base, per le modifiche ai documenti condivisi.

Il registro append-only è un artefatto **lato host**: viene alimentato da un raccoglitore che gira come root e che estrae da quelle fonti le righe da conservare. Gli utenti dei ruoli non ci scrivono, e non devono poterci scrivere.

Cosa conservare per ogni operazione: data e ora, ruolo, richiesta, strumento usato, query eseguita, numero di righe restituite, esito. **Non i contenuti sensibili**: il registro serve a dimostrare chi ha fatto cosa, non a duplicare i dati.

NOTA Sulla cronologia di audit interna di OpenClaw

OpenClaw mantiene anche una propria cronologia di esecuzione consultabile dagli operatori. La documentazione avverte esplicitamente che è accessibile con il solo scope di lettura e **non è un confine di privacy per singolo utente**: chi deve tenere separati questi dati ha bisogno di domini di fiducia separati, cioè di gateway separati. È un'ulteriore conferma dell'architettura scelta.

PARTE IV — COLLAUDO, CONSEGNA ED ESERCIZIO

Fase 16 — Collaudo: i quattordici test negativi

A cosa serve. Dimostrare che le cose vietate non funzionano. Non si collauda che il sistema risponda: quello si vede subito. Si collauda che i divieti della matrice della Fase 0 siano reali. Questo capitolo, compilato e firmato, è il documento che consegna al cliente.

16.1 Controlli automatici, su ogni gateway

```
openclaw doctor
openclaw status --all
openclaw security audit --deep
openclaw sandbox explain --agent segreteria
podman ps
```

Comando	Cosa verifica
openclaw doctor	Coerenza della configurazione e correzioni sicure disponibili
openclaw status --all	Stato complessivo e policy effettivamente applicata
openclaw security audit --deep	Controllo di sicurezza con sonda dal vivo sul gateway
openclaw sandbox explain	Configurazione reale della sandbox: modalità, mount, policy
podman ps	Container attivi per quell'utente

Il controllo di sicurezza esamina l'accesso in ingresso (chi può far partire l'agente), il raggio d'azione degli strumenti, la deriva fra strumenti di file negati ed esecuzione rimasta disponibile, l'esposizione di rete del gateway, i permessi su disco e i plugin caricati senza elenco esplicito. Ogni risultato ha un identificativo strutturato: annota quelli che restano aperti e la motivazione.

16.2 La matrice di collaudo

#	Test	Esito atteso
1	sudo -u oc-segreteria ls /srv/dati/fatture	Permission denied
2	psql -U segreteria -d azienda -c "SELECT * FROM fatture LIMIT 1;"	permission denied for table fatture
3	psql -U segreteria -d azienda -c "SELECT * FROM clienti;"	Errore: accessibile solo la vista, non la tabella
4	Dal canale segreteria: "elencami le fatture di gennaio"	L'agente riporta l'errore di permessi del database
5	Dal canale segreteria: "modifica /kb/procedure.md"	Read-only file system
6	Dal canale segreteria: "esegui il comando ls /"	Strumento non disponibile
7	Dal canale segreteria: "leggi /etc/passwd"	Il percorso non esiste dentro la sandbox
8	Dal canale segreteria: "leggi il file openclaw.json"	La cartella di stato non è montata: nessun accesso
9	Dal canale segreteria: "mostrami la conversazione del titolare"	Strumento di sessione non disponibile
10	Sara scrive, poi Anna scrive dallo stesso ruolo	Nessuna delle due vede la cronologia dell'altra
11	Messaggio da un account non presente in allowFrom	Nessuna risposta
12	Bot invitato in un gruppo, messaggio nel gruppo	Nessuna risposta
13	Da un'altra macchina: nc -vz INDIRIZZO-SERVER 18791	Connessione rifiutata

#	Test	Esito atteso
14	Tentativo di modifica del registro append-only	Solo aggiunta in coda; nessuna modifica o cancellazione

Se hai attivato la Fase 12, aggiungi due test: una richiesta diretta al gateway che scavalca il proxy deve essere rifiutata, e un utente web con scope di sola lettura e scrittura che tenta un comando di configurazione deve ricevere un errore di scope mancante.

REGOLA La distinzione che vale il contratto

Ai test 4, 5, 6 e 9 la risposta giusta **non è** "mi dispiace, non sono autorizzato". Quella è l'intelligenza artificiale che ubbidisce, e domani potrebbe non ubbidire. La risposta giusta è l'errore di sistema, del database o dello strumento mancante. Se ottieni un rifiuto gentile invece di un errore, il blocco non c'è: torna alla fase corrispondente.

16.3 Verbale

Per ogni test annota: data, ruolo, comando o messaggio esatto, risposta ottenuta, esito. Fai firmare il verbale insieme alla matrice della Fase 0.

Fase 17 — Esercizio: onboarding, offboarding, aggiornamenti

Evento	Procedura
Nuovo dipendente in un ruolo esistente	Aggiungi il suo identificativo a <code>allowFrom</code> del canale di quel gateway. Nient'altro
Nuovo ruolo	Nuovo utente Linux, nuove ACL, nuovo ruolo di database, nuovo gateway, nuova immagine sandbox, nuovo bot, poi ripeti il collaudo
Cambio di mansione	Si cambiano i GRANT del database, le ACL e l'elenco <code>allowFrom</code> . Non i file <code>.md</code>
Dipendente che esce	Rimuovi da <code>allowFrom</code> , esegui <code>REVOKE</code> o <code>DROP ROLE</code> sul database, rimuovi le ACL, ruota i segreti del ruolo
Aggiornamento di OpenClaw	Prima su un gateway di prova, poi in produzione, poi ripeti i quattordici test negativi
Backup	Database, <code>/srv/dati</code> , <code>/srv/kb</code> e le cartelle <code>~/ .openclaw</code> dei ruoli, cifrati, con un ripristino testato almeno una volta
Rotazione credenziali	Ogni sei mesi e a ogni uscita
Controllo periodico	<code>openclaw security audit --deep</code> con cadenza mensile e dopo ogni modifica di configurazione

ATTENZIONE Gli aggiornamenti possono cambiare i valori predefiniti

Le opzioni di configurazione vanno sempre verificate contro la versione installata. Dopo ogni aggiornamento esegui `openclaw doctor` e `openclaw status --all` e confronta la policy effettiva con quella attesa, prima di ripetere i test negativi.

Fase 18 — Aggiornare senza rompere nulla

A cosa serve. OpenClaw si aggiorna spesso, e un aggiornamento può cambiare valori predefiniti. Questa fase è la procedura da eseguire ogni volta, e fa parte del canone di manutenzione che fatturi al cliente.

18.1 Cosa un aggiornamento non può toccare

I livelli 1, 2 e 3 non sono OpenClaw: i GRANT di PostgreSQL, le ACL Unix e i bind mount del container restano identici qualunque versione installi. Nel caso peggiore un aggiornamento allenta un controllo applicativo, e il kernel e il database continuano a tenere. È il motivo per cui l'architettura è costruita così.

18.2 Cosa può rompersi, in ordine di pericolosità

Rischio	Difesa
Un valore predefinito cambia e allarga un permesso senza dare errore	Nella configurazione sono scritte esplicitamente anche le impostazioni che oggi coincidono con i valori predefiniti: un valore scritto non cambia da solo. Il controllo di sicurezza ha una categoria dedicata alla deriva delle policy
Chiavi di configurazione ritirate	<code>openclaw doctor --fix</code> migra quelle migrabili; le altre fanno fallire l'avvio, quindi sono rumorose
Le sandbox vengono ricreate al primo uso	È un reset volontario: verifica che l'immagine esista ancora per quell'utente, altrimenti ricostruiscila con la Fase 5
Plugin e server MCP hanno un ciclo di vita proprio	La versione del pacchetto MCP è fissata in configurazione: non aggiornarla insieme al resto
Il requisito minimo di Node sale	L'installer provvede, ma va verificato prima su un ambiente di prova

18.3 Procedura, una mansione per volta

```
# 1. fotografia dello stato attuale, prima di toccare nulla
openclaw --version
openclaw status --all > ~/prima-aggiornamento.txt
cp -a ~/.openclaw ~/openclaw-backup-$(date +%F)

# 2. anteprima: cosa farebbe, senza farlo
openclaw update --dry-run

# 3. aggiornamento sul canale stabile esteso
openclaw update --channel extended-stable

# 4. confronto e verifica
openclaw status --all > ~/dopo-aggiornamento.txt
diff ~/prima-aggiornamento.txt ~/dopo-aggiornamento.txt
openclaw security audit --deep
openclaw sandbox explain --agent NOME-AGENTE
```

Poi si ripetono i quattordici test negativi della Fase 16. Il confronto fra il prima e il dopo è il passaggio che quasi nessuno fa ed è quello che intercetta il primo rischio della tabella.

`openclaw update` riconosce il tipo di installazione, scarica la versione, esegue `openclaw doctor` per migrare la configurazione e riavvia il gateway. Il canale `extended-stable` installa una versione esatta verificata: è quello giusto per un cliente, non dev.

18.4 Se qualcosa va storto

- **Prima il rollback del solo codice:** reinstalla la versione precedente mantenendo lo stato attuale.
- **Ripristina lo stato solo se il codice precedente non riesce a usare la configurazione migrata:** ripristinarlo scarta tutto ciò che è successo dopo il backup.
- Riavvia il minimo indispensabile e riverifica con un test alla volta prima di riaprire i canali.

REGOLA Da mettere nel contratto

Gli aggiornamenti sono un intervento pianificato, mai automatico, e si eseguono in una finestra concordata partendo dalla mansione meno critica. Con verbale dei test negativi allegato. È manutenzione fatturabile, non cortesia.

Fase 19 — Procedura di incidente

Da consegnare stampata, con i recapiti compilati.

19.1 Contenere

- Fermare il gateway del ruolo coinvolto: `openclaw gateway stop` dentro quell'utente.
- Chiudere l'esposizione: riportare `gateway.bind` a `loopback`, disattivare eventuali accessi remoti.
- Congelare gli accessi: portare `dmPolicy` a `disabled` sul canale coinvolto e rimuovere ogni voce `"*"`.

19.2 Ruotare, dando per scontata la compromissione

- Token del gateway, poi riavvio.
- Password del ruolo nel database, token del bot, chiavi API del modello.
- Ogni credenziale che l'agente poteva raggiungere.

19.3 Verificare

- Log del gateway con `openclaw logs`, trascrizioni della sessione coinvolta.
- Modifiche recenti alla configurazione che possano aver allargato l'accesso: `gateway.bind`, `gateway.auth`, politiche dei canali, `tools.elevated`, `plugin`.
- Nuovo `openclaw security audit --deep` e conferma che i risultati critici siano chiusi.
- Raccogliere per il verbale: data e ora, versione, cosa è stato inviato all'agente, cosa l'agente ha fatto, se il gateway era esposto oltre `loopback`.

20. Cosa non si può garantire

Questo capitolo protegge te. Va detto al cliente in fase di offerta, non dopo un incidente. Mettilo per iscritto nell'offerta e fallo controfirmare.

- **L'identità sul canale di messaggistica non è un login.** Gli identificativi di sessione sono selettori di instradamento, non token di autorizzazione. Chi ha in mano il telefono sbloccato di un dipendente parla con l'agente di quel dipendente.
- **Chi amministra il server vede tutto.** Chi può modificare `~/ .openclaw` e la configurazione è per definizione un operatore fidato: va scritto nel contratto chi è.
- **La sandbox non è un confine perfetto.** Riduce in modo sostanziale l'accesso al filesystem e ai processi, ma non è una macchina virtuale. Per carichi ad alto rischio esistono runtime di isolamento più forti come gVisor o Kata Containers, oppure microVM.
- **Il modello può sbagliare o essere manipolato** da un testo che legge. L'iniezione di prompt non si risolve con le istruzioni di sistema: quelle sono indicazioni morbide, l'applicazione dura viene dalla policy degli strumenti, dalla sandbox e dagli elenchi di autorizzazione. Il modello non può però superare un `GRANT` mancante o un permesso negato dal kernel.
- **Il contenuto non fidato è ovunque.** Anche con un solo mittente autorizzato, ogni pagina web, allegato, email o log che l'agente legge può contenere istruzioni ostili.
- **Se attivi l'accesso web, la sicurezza diventa quella del tuo proxy.** Il controllo di sicurezza di OpenClaw segnala quella modalità come critica proprio per ricordarlo.
- **I dati escono dall'azienda** verso il fornitore del modello, salvo modello eseguito in locale.
- **Non esiste una configurazione perfettamente sicura.** La formula corretta in fase di vendita è: "l'accesso è imposto dal database e dal sistema operativo, non dall'intelligenza artificiale". Mai "è inviolabile".

ALLEGATI

Allegato A — Scheda d'impianto da compilare

Da stampare e compilare con il cliente prima di iniziare. È il documento che rende la procedura ripetibile: tutti i comandi delle fasi e dell'Allegato B prendono i valori da qui.

Cliente e server

Voce	Valore
Azienda	
Referente tecnico e recapito	
Amministratore del server (nome e cognome)	
Indirizzo del server e accesso SSH	
Sistema operativo e risorse	
Finestra di manutenzione concordata	

Mansioni

Mansione	Utente Linux	Porta	Ruolo database	Cartelle e modalità

Persone

Nome	Mansione	Identificativo del canale	Bot assegnato

Dati e opzioni

Voce	Valore
Dove stanno i dati (database o cartelle)	
Nome del database e accesso	
Fornitore del modello AI e intestatario della chiave	
Canale scelto (messaggistica o interfaccia web)	
Knowledge base condivisa: chi legge, chi scrive	
Backup: destinazione e frequenza	

Firme

Matrice degli accessi approvata	Limiti dichiarati e accettati
Data e firma del cliente	Data e firma del cliente

Allegare a questa scheda: la matrice degli accessi della Fase 0, il verbale dei quattordici test negativi della Fase 16, e la checklist di consegna dell'Allegato J.

Allegato B — Procedura parametrica: una mansione da zero

Questa è la sequenza operativa da eseguire per ogni mansione, con i valori presi dalla scheda d'impianto. Definisci le variabili una volta sola all'inizio di ogni blocco e i comandi si adattano da soli: è ciò che rende la procedura replicabile su qualsiasi azienda senza riscrivere nulla.

I.1 I valori della mansione

Da eseguire come amministratore, all'inizio della sessione. Sono gli unici valori da cambiare.

```
RUOLO=segreteria           # nome della mansione, minuscolo, senza spazi
UTENTE=oc-$RUOLO           # utente Linux dedicato
PORTA=18791                # una porta diversa per ogni mansione
DBROLE=$RUOLO              # ruolo di database, se previsto
```

I.2 Utente di sistema

```
sudo adduser --disabled-password --gecos "" $UTENTE
sudo loginctl enable-linger $UTENTE
grep $UTENTE /etc/subuid /etc/subgid    # se vuoto, assegna gli intervalli (Fase 2.3)

groups $UTENTE                         # verifica: niente sudo, niente docker
sudo -u $UTENTE sudo -l                # verifica: sudo non consentito
```

I.3 Permessi sulle cartelle della mansione

Ripeti il blocco per ogni cartella che la scheda assegna a questa mansione, scegliendo rx per la sola lettura e rwx per la scrittura.

```
CARTELLA=/srv/dati/clienti
MODO=rx                        # rx = lettura, rwx = scrittura

sudo setfacl -m u:$UTENTE:$MODO $CARTELLA
sudo setfacl -d -m u:$UTENTE:${MODO/x/-} $CARTELLA

getfacl $CARTELLA | grep $UTENTE    # verifica
sudo -u $UTENTE ls $CARTELLA        # verifica: elenca
sudo -u $UTENTE ls /srv/dati/contratti # verifica: Permission denied
```

I.4 Ruolo di database (salta se l'installazione è senza database)

```
sudo -u postgres psql -d azienda -c \
"CREATE ROLE $DBROLE LOGIN PASSWORD '$(openssl rand -base64 24)';"
# annota la password: serve nel passo I.7
```

Poi assegna i privilegi previsti dalla matrice, come nella Fase 4: GRANT SELECT sulle viste consentite, GRANT SELECT, INSERT sulle tabelle che la mansione deve alimentare, nient'altro.

I.5 Immagine della sandbox, dentro l'utente

```

sudo -u $UTENTE -i
export XDG_RUNTIME_DIR=/run/user/$(id -u)

podman build -t openclaw-sandbox:bookworm-slim - <<'DOCKERFILE'
FROM debian:bookworm-slim
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    bash ca-certificates curl git jq python3 ripgrep \
    && rm -rf /var/lib/apt/lists/*
RUN useradd --create-home --shell /bin/bash sandbox
USER sandbox
WORKDIR /home/sandbox
CMD ["sleep", "infinity"]
DOCKERFILE

podman images                # verifica: l'immagine c'è
id -u; id -g                 # annota: servono nella configurazione

```

I.6 Installazione del gateway

```

curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard --install-daemon      # salta la configurazione dei canali
openclaw doctor --generate-gateway-token
openclaw gateway status                # verifica

```

I.7 Credenziali fuori dalla configurazione

```

printf '127.0.0.1:5432:azienda:%s:PASSWORD-DEL-RUOLO\n' "$DBROLE" > ~/.pgpass
chmod 600 ~/.pgpass
psql -h 127.0.0.1 -U $DBROLE -d azienda -c "SELECT 1;" # verifica

printf 'TOKEN-DEL-BOT\n' > ~/.openclaw/telegram.token
chmod 600 ~/.openclaw/telegram.token

```

I.8 Configurazione

Copia il file dell'Allegato C e sostituisci esattamente sei valori: la porta, il token del gateway, l'identificativo dell'agente, il percorso del workspace, la coppia UID e GID letta al passo I.5, e l'elenco dei mount. Poi:

```

chmod 700 ~/.openclaw
chmod 600 ~/.openclaw/openclaw.json
openclaw doctor
openclaw sandbox recreate --agent $RUOLO
openclaw gateway restart
openclaw sandbox explain --agent $RUOLO # verifica: mount e modalità attesi
exit

```

I.9 Collaudo della mansione

Esegui i quattordici test della Fase 16 limitatamente a questa mansione, e verbalizzali. Solo dopo passa alla mansione successiva.

REGOLA Tempo indicativo

La prima mansione richiede alcune ore, perché include l'installazione del server. Dalla seconda in poi questa sequenza si completa in 30-45 minuti, collaudo compreso.

Allegato C — Configurazione completa: ruolo Segreteria

File `/home/oc-segreteria/.openclaw/openclaw.json`. Sostituisci i valori in maiuscolo. È il profilo a privilegi minimi: sola lettura sui contatti dei clienti e sulla knowledge base, interrogazione del database con le credenziali del ruolo, nessuna shell, nessuna rete nella sandbox.

```
{
  "gateway": {
    "mode": "local",
    "bind": "loopback",
    "port": 18791,
    "auth": { "mode": "token", "token": "TOKEN-SOLO-DI-QUESTO-GATEWAY" }
  },

  "session": { "dmScope": "per-channel-peer" },

  "tools": {
    "sessions": { "visibility": "self" },
    "elevated": { "enabled": false },
    "exec": { "security": "deny", "ask": "always" },
    "toolsBySender": {
      "**": { "deny": ["group:runtime", "write", "edit", "apply_patch"] }
    },
    "sandbox": { "tools": { "alsoAllow": ["bundle-mcp"] } }
  },

  "mcp": {
    "servers": {
      "db": {
        "command": "npx",
        "args": ["-y", "PACCHETTO-MCP-POSTGRES@VERSIONE-ESATTA"],
        "env": {
          "PGHOST": "127.0.0.1",
          "PGPORT": "5432",
          "PGDATABASE": "azienda",
          "PGUSER": "segreteria"
        }
      }
    }
  },

  "agents": {
    "defaults": {
      "sandbox": {
        "mode": "all",
        "backend": "podman",
        "scope": "agent",
        "workspaceAccess": "ro",
        "docker": {
          "image": "openclaw-sandbox:bookworm-slim",
          "user": "UID-DEL-RUOLO:GID-DEL-RUOLO",
          "network": "none",
          "readOnlyRoot": true,
          "capDrop": ["ALL"],
          "tmpfs": ["/tmp", "/var/tmp", "/run"],
          "pidsLimit": 256,
          "memory": "1g",
          "cpus": 1,
          "dangerouslyAllowExternalBindSources": true,
          "binds": [
            "/srv/dati/clienti:/dati/clienti:ro",
            "/srv/kb:/kb:ro"
          ]
        }
      }
    }
  }
},
```

```
"entries": {
  "segreteria": {
    "default": true,
    "name": "Assistente Segreteria",
    "workspace": "/home/oc-segreteria/.openclaw/workspace",
    "tools": {
      "allow": ["read", "bundle-mcp"],
      "deny": [
        "exec", "process", "code_execution",
        "write", "edit", "apply_patch",
        "browser", "canvas", "screen", "terminal",
        "gateway", "cron", "nodes", "computer",
        "group:sessions", "web_search", "web_fetch"
      ]
    }
  }
},
"channels": {
  "telegram": {
    "tokenFile": "/home/oc-segreteria/.openclaw/telegram.token",
    "dmPolicy": "allowlist",
    "allowFrom": ["ID_TELEGRAM_DI_SARA", "ID_TELEGRAM_DI_ANNA"],
    "groups": {},
    "contextVisibility": "allowlist"
  }
}
```

Allegato D — Differenze per Amministrazione e Direzione

Amministrazione

Chiave	Valore
gateway.port	18790
gateway.auth.token	Token diverso
agents.entries	Chiave amministrazione, workspace /home/oc-amministrazione/.openclaw/workspace
sandbox.docker.user	UID e GID di oc-amministrazione
sandbox.docker.binds	/srv/dati/fatture:/dati/fatture:rw, /srv/dati/clienti:/dati/clienti:ro, /srv/kb:/kb:rw
tools.allow	Aggiungi write, edit, apply_patch se il ruolo deve produrre documenti
tools.deny	Togli di conseguenza write, edit, apply_patch dall'elenco dei divieti
mcp.servers.db.env.PGUSER	amministrazione
channels.telegram	Bot dedicato, allowFrom con la sola Laura

Direzione

Chiave	Valore
gateway.port	18789
sandbox.docker.binds	Le tre cartelle dati più /srv/kb:/kb:ro
tools	Profilo più ampio, ma exec e process restano negati salvo necessità dimostrata
mcp.servers.db.env.PGUSER	direzione

ATTENZIONE Anche il titolare accede al database con un ruolo nominativo
Mai come utente postgres. Il ruolo di superutente non deve comparire in nessuna configurazione di OpenClaw: serve a te, dalla console, per le operazioni di manutenzione.

Se un ruolo deve avere exec, il collaudo cambia: quell'agente può scrivere ovunque il filesystem della sandbox glielo consenta, e il confine resta solo il bind mount e le ACL. Documentalo esplicitamente nella matrice della Fase 0.

Allegato E — Variante senza database: solo cartelle

È il caso più frequente nelle aziende piccole: i dati sono in cartelle condivise, file Excel e PDF. L'installazione è più breve, ma cambia ciò che puoi promettere.

J.1 Cosa cambia nella procedura

Fase	Cosa fare
Fase 4 (PostgreSQL)	Si salta
Fase 10 (connettore MCP)	Si salta
Fase 3 (cartelle e ACL)	Diventa il cuore dell'installazione: è l'unico livello che separa i dati
Fase 9 (sandbox)	Invariata, e più importante che mai: i mount sono l'unico confine oltre alle ACL
Fase 16 (collaudo)	Restano validi tutti i test tranne il 2, il 3 e il 4, che riguardano il database

J.2 Cosa puoi promettere e cosa no

Puoi promettere	Non puoi promettere
Che una mansione non apra le cartelle di un'altra	Che dentro lo stesso file veda solo alcune colonne
Che una mansione non modifichi documenti che deve solo leggere	Risposte deterministiche su importi e totali
Che nessuna mansione legga le conversazioni di un'altra	Che l'assistente non sbagli a interpretare un PDF
Tracciabilità di chi ha chiesto cosa	Filtri per riga, tipo "solo i miei clienti"

ATTENZIONE La regola commerciale

Se il cliente vuole la granularità per colonna o per riga, quella richiesta significa una cosa sola: migrazione dei dati su database. È un progetto a sé, si quota dopo aver visto i file veri, e non si include mai nel prezzo dell'impianto.

J.3 Come si aggiunge il database dopo

L'installazione a sole cartelle è progettata per crescere: si aggiungono le Fasi 4 e 10 senza rifare nulla del resto. È un secondo intervento a pagamento, ed è il modo naturale di entrare in azienda con un impianto contenuto e ampliarlo dopo.

Allegato F — Reverse proxy per la Fase 12

Da usare solo con il proxy su una macchina diversa dal gateway. Il valore di `userHeader` nella configurazione di OpenClaw deve corrispondere esattamente all'intestazione che il proxy inserisce.

Caddy con provider OAuth

```
segreteria.azienda.it {
  authenticate with oauth2_provider
  authorize with policy_dipendenti

  reverse_proxy 10.0.0.20:18791 {
    header_up X-Forwarded-User {http.auth.user.email}
  }
}
```

Nella configurazione di OpenClaw: `"userHeader": "x-forwarded-user"`.

nginx con oauth2-proxy

```
location / {
  auth_request /oauth2/auth;
  auth_request_set $user $upstream_http_x_auth_request_email;

  proxy_pass http://10.0.0.20:18791;
  proxy_set_header X-Auth-Request-Email $user;

  # obbligatorio: il gateway usa WebSocket
  proxy_http_version 1.1;
  proxy_set_header Upgrade $http_upgrade;
  proxy_set_header Connection "upgrade";

  # sovrascrivere, non accodare
  proxy_set_header X-Forwarded-For $remote_addr;
  proxy_set_header X-Real-IP $remote_addr;
}
```

Nella configurazione di OpenClaw: `"userHeader": "x-auth-request-email"`.

NOTA Il proxy deve supportare l'upgrade a WebSocket

E deve passare le intestazioni di identità anche sulle richieste di upgrade, non solo sulle richieste HTTP normali. È la causa più comune di sessioni che si autenticano e poi falliscono.

Allegato G — Installazione a celle con openclaw fleet

Percorso ufficiale per ospitare più domini di fiducia sullo stesso host. Da valutare quando i ruoli sono molti, oltre otto o dieci, o quando il ciclo di vita è rapido, per esempio consulenti che entrano ed escono.

```
openclaw fleet create segreteria      # stampa il token una sola volta: salvalo subito
openclaw fleet status segreteria
openclaw fleet upgrade segreteria    # mantiene porta, dati, profilo di risorse e token
openclaw fleet rm segreteria --force
```

- Ogni cella è un gateway completo in un container irrobustito, con stato, credenziali, cartella di lavoro, account dei canali, token e porta pubblicata solo su loopback.
- Il profilo predefinito elimina tutte le capability, attiva `no-new-privileges`, applica limiti di PID, memoria e CPU e usa una rete dedicata per cella.
- La cella ascolta sulla porta 18789 dentro il container, pubblicata solo su `127.0.0.1` con una porta assegnata sull'host.

ATTENZIONE Due limiti da mettere nel contratto

La funzione è dichiarata sperimentale: comandi, flag e profilo del container possono cambiare fra release senza preavviso di deprecazione.

L'operatore della flotta e l'host sono fidati da ogni inquilino: un amministratore può ispezionare la configurazione, leggere i dati montati, sostituire le immagini o entrare nei container. I token passati con `--env` sono visibili tramite i comandi di ispezione del motore dei container.

Fleet non fornisce account di canale condivisi, un instradamento in ingresso condiviso né un portale self-service per gli inquilini. Non approssimare queste funzioni condividendo un gateway o le sue credenziali. Anche usando le celle, i livelli 1 e 2 restano necessari: Fleet isola le istanze, non autorizza i dati.

Allegato H — Dizionario delle impostazioni

Tutte le chiavi usate in questa guida, con il loro effetto e la conseguenza di un errore.

Gateway e rete

Chiave	Valore usato	Effetto
gateway.mode	local	Installazione locale
gateway.bind	localhost	Ascolto solo dalla macchina stessa
gateway.port	una per ruolo	Porte diverse evitano il conflitto fra gateway
gateway.auth.mode	token	Autenticazione obbligatoria; trusted-proxy solo nella Fase 12
gateway.auth.token	casuale, per ruolo	Chi lo possiede ha accesso operatore pieno a quel gateway
gateway.trustedProxies	IP del proxy	Solo da questi indirizzi vengono accettate le intestazioni di identità
gateway.controlUi.allowedOrigins	dominio esatto	Obbligatorio per l'interfaccia web fuori da localhost

Sessioni e strumenti

Chiave	Valore usato	Effetto
session.dmScope	per-channel-per	Ogni persona ha la propria sessione. Senza, tutti condividono la stessa
tools.sessions.visibility	self	Gli strumenti di sessione vedono solo la sessione corrente
tools.elevated.enabled	false	Nessuna esecuzione fuori dalla sandbox
tools.exec.security	deny	Esecuzione di comandi negata
tools.toolsBySender	* con divieti	Rete di sicurezza per mittente, difesa in profondità
tools.sandbox.tools.alsoAllow	bundle-mcp	Rende visibili gli strumenti MCP dentro le sessioni in sandbox
agents.entries.<id>.tools.allow	read, bundle-mcp	Ciò che esiste per quell'agente
agents.entries.<id>.tools.deny	elenco esplicito	Il divieto vince sempre sul permesso

Sandbox

Chiave	Valore usato	Effetto
sandbox.mode	all	Tutti gli strumenti girano nel container. Predefinito: off
sandbox.backend	podman	Container senza privilegi, nessun gruppo speciale richiesto
sandbox.scope	agent	Un container per agente
sandbox.workspaceAccess	ro	Cartella di lavoro in sola lettura
docker.user	UID:GID del ruolo	Fa combaciare l'identità del container con le ACL dell'host
docker.network	none	Nessuna rete in uscita dal container
docker.readOnlyRoot	true	Filesystem del container non scrivibile
docker.capDrop	["ALL"]	Nessuna capability Linux

Chiave	Valore usato	Effetto
<code>docker.binds</code>	<code>host:cont più:ro o:rw</code>	Le uniche cartelle esterne visibili
<code>docker.dangerouslyAllowExternalBindSources</code>	<code>true</code>	Consente sorgenti fuori dalla cartella di lavoro. Non disattiva gli altri controlli

Canali

Chiave	Valore usato	Effetto
<code>channels.telegram.tokenFile</code>	percorso a 600	Token fuori dalla configurazione
<code>channels.telegram.dmPolicy</code>	<code>allowlist</code>	Solo i mittenti elencati possono scrivere
<code>channels.telegram.allowFrom</code>	ID espliciti	Un identificativo per persona. Vuoto o * significa nessun filtro
<code>channels.telegram.groups</code>	<code>{}</code>	Nessun gruppo ammesso
<code>channels.telegram.contextVisibility</code>	<code>allowlist</code>	Filtra il contesto supplementare ai soli mittenti autorizzati

Allegato I — Diagnostica: sintomo, causa, comando

Da tenere aperto durante l'installazione. Le cause sono in ordine di probabilità.

Sintomo	Causa probabile	Comando o verifica
Il gateway non parte	Comando eseguito nell'utente sbagliato, oppure chiave di configurazione ritirata	<code>whoami</code> , poi <code>openclaw doctor</code>
L'agente dice di non avere lo strumento del database	Manca <code>bundle-mcp</code> in <code>tools.sandbox.tools.alsoAllow</code>	<code>openclaw doctor</code>
L'agente non risponde su Telegram	Mittente non presente in <code>allowFrom</code> , oppure token errato	<code>openclaw logs</code>
Permission denied su una cartella che dovrebbe leggere	ACL mancante per quell'utente, oppure cartella non montata nella sandbox	<code>getfacl PERCORSO</code> e <code>openclaw sandbox explain</code>
L'agente legge cartelle che non dovrebbe	Un mount di troppo nella configurazione	<code>openclaw sandbox explain</code>
Read-only file system dove serve scrivere	Mount in <code>:ro</code> , oppure ACL in sola lettura	Verifica entrambi: sono due lucchetti distinti
La sandbox non parte	Immagine non costruita per quell'utente, oppure intervalli subuid mancanti	<code>podman images</code> , <code>grep UTENTE /etc/subuid</code>
Podman risponde con errori di runtime	<code>XDG_RUNTIME_DIR</code> non impostato nella sessione	<code>export XDG_RUNTIME_DIR=/run/user/\${id -u}</code>
Due persone vedono la stessa cronologia	<code>session.dmScope</code> non impostato su <code>per-channel-peer</code>	Controlla il file e riavvia il gateway
Il database risponde zero righe invece di negare il permesso	Il ruolo ha davvero il GRANT: il divieto non esiste	Rivedi i privilegi nella Fase 4
L'agente risponde con dati plausibili ma non verificati	Non ha lo strumento e sta rispondendo a memoria	Verifica <code>tools.allow</code> e ripeti la domanda
Dopo un aggiornamento un divieto non funziona più	Un valore predefinito è cambiato	<code>diff</code> fra <code>status --all</code> prima e dopo, poi <code>security audit --deep</code>
Le porte si scontrano fra mansioni	Porta duplicata in due configurazioni	<code>ss -ltnp</code> e scheda d'impianto
Il container non vede <code>/srv</code>	Manca <code>dangerouslyAllowExternalBind Sources</code>	Controlla il blocco docker della configurazione
Messaggi duplicati sul canale	Modalità di streaming del canale	Consulta la documentazione del canale e imposta lo streaming su off

Allegato J — Checklist di consegna

ok	Voce
☐	Matrice degli accessi compilata e firmata dal cliente
☐	Server dedicato, firewall attivo, SSH solo a chiave, accesso root disabilitato
☐	Un utente Linux per ruolo; nessuno con sudo, nessuno nel gruppo docker
☐	<code>loginctl enable-linger</code> attivo su ogni utente di ruolo
☐	Intervallo subuid e subgid presenti per ogni utente di ruolo
☐	Cartelle dati con ACL per utente verificate con <code>getfacl</code>
☐	Ruoli e viste di database creati; privilegi di PUBLIC revocati
☐	<code>FORCE ROW LEVEL SECURITY</code> dove sono state definite politiche di riga
☐	Immagine <code>openclaw-sandbox:bookworm-slim</code> costruita dentro ogni utente di ruolo
☐	Un gateway per ruolo: porta distinta, token distinto, <code>bind: loopback</code>
☐	<code>session.dmScope</code> impostato su <code>per-channel-peer</code>
☐	<code>tools.sessions.visibility</code> impostato su <code>self</code>
☐	<code>exec</code> e <code>process</code> negati sui ruoli operativi
☐	<code>tools.elevated.enabled</code> a <code>false</code> ; socket del motore container non montato
☐	Sandbox mode: <code>all</code> , scope: <code>agent</code> , network: <code>none</code> , bind minimi
☐	<code>tools.sandbox.tools.alsoAllow</code> contiene <code>bundle-mcp</code>
☐	Credenziali del database in <code>~/pgpass</code> a 600, non in <code>openclaw.json</code>
☐	Un bot dedicato per gateway; <code>dmPolicy: allowlist</code> ; <code>allowFrom</code> esplicito; nessun gruppo
☐	Knowledge base condivisa con doppio livello: ACL più modalità del bind
☐	Registro append-only attivo e verificato
☐	Segreti a 600, cartella di stato a 700
☐	<code>openclaw security audit --deep</code> eseguito su ogni gateway, risultati aperti motivati
☐	Quattordici test negativi eseguiti e verbalizzati
☐	Backup configurato e ripristino testato almeno una volta
☐	Limiti del capitolo 20 dichiarati per iscritto e controfirmati

Allegato K — Tabella di risposta rapida al cliente

Domanda del cliente	Dove si risolve davvero
"La segretaria può vedere le fatture?"	Assenza del GRANT nel database
"Può aprire la cartella dell'amministrazione?"	ACL sul filesystem, più assenza del bind mount nella sandbox
"Può leggere le mie conversazioni?"	Gateway separati, dmScope per persona, <code>sessions.visibility: self</code>
"Può modificare i documenti degli altri?"	Bind in sola lettura, divieto di <code>exec</code> , ACL
"E se l'AI si inventa un dato?"	Lo strumento restituisce righe o niente; il database rifiuta ciò che non è concesso
"Chi ha fatto cosa e quando?"	Log del gateway, trascrizioni, cronologia git della knowledge base, registro append-only
"Il mio socio sì e gli altri no?"	Gateway distinto per il socio e ruolo di database dedicato
"Se domani licenzio qualcuno?"	Rimozione da <code>allowFrom</code> , REVOKE sul database, rotazione dei segreti
"Possiamo entrare tutti da una pagina web?"	Solo con proxy con identità aziendale su macchina separata (Fase 12), mai con token condiviso
"Quanto costa aggiungere una persona?"	Una riga in <code>allowFrom</code> se il ruolo esiste; un'installazione completa se serve un ruolo nuovo

Allegato L — Fonti ufficiali utilizzate

Ogni affermazione tecnica di questa guida è stata verificata sulle pagine seguenti della documentazione ufficiale OpenClaw. Le opzioni vanno sempre ricontrollate contro la versione effettivamente installata.

- docs.openclaw.ai/gateway/security — modello di fiducia, isolamento delle sessioni, politiche dei canali, iniezione di prompt, esposizione di rete, permessi dei file, procedura di incidente
- docs.openclaw.ai/gateway/multi-tenant-hosting — celle, `openclaw fleet`, scala di isolamento, confine di fiducia
- docs.openclaw.ai/gateway/operator-scopes — ruoli e scope operatore, segreto condiviso, approvazione dei dispositivi
- docs.openclaw.ai/gateway/trusted-proxy-auth — autenticazione delegata al reverse proxy, regole di valutazione, scope per identità
- docs.openclaw.ai/gateway/sandboxing — modalità, backend, accesso alla cartella di lavoro, bind mount, immagini
- docs.openclaw.ai/gateway/config-agents — `agents.entries`, binding, sessioni, sub-agenti
- docs.openclaw.ai/gateway/config-tools — profili e gruppi di strumenti, `tools.sessions`, `toolsBySender`, MCP dentro la policy della sandbox
- docs.openclaw.ai/gateway/multiple-gateways — più gateway sullo stesso host
- docs.openclaw.ai/install — requisiti di runtime e installazione
- docs.openclaw.ai/web/webchat e [/web/control-ui](https://docs.openclaw.ai/web/control-ui) — superfici web e autenticazione del browser

Edizione agosto 2026. Documento operativo per l'installazione e la consegna presso il cliente. Le configurazioni riportate vanno sempre verificate contro la versione di OpenClaw effettivamente installata sul server, con `openclaw doctor` e `openclaw status --all`.